

claude-fable_Einstein-Rosen-2-Planes

Pre-gravity, Pre-Big-Bang: 3 Generations of Einstein-Rosen 2-Plane Bridges

Frame fields, spin connections and the 16-component split-octonion spinor on a curved 4+4 spacetime

Refactored, commented and optimized from the author's scratch notebook
"Pre-gravityPre-Big_Bang_M6=3-Generations_of_Einstein-Rosen-2-Planes.nb".

Original mathematics and physics: Patrick L. Nash, Ph.D. (c) 2022, under the GNU General Public License.

Professor, UTSA Physics and Astronomy, Retired. Patrick299Nash at gmail.

Please cite that work, and this web page, if you use it.

Refactoring, the frame-field / spin-connection chapters, and the three new bridges:
prepared with Claude (Anthropic) at the author's direction, 2026-09-14.

WARNING (from the original): syncope, presyncope AHEAD.

Whenever possible we follow ROBERT L. BRYANT, "SUBMANIFOLDS AND SPECIAL STRUCTURES ON THE OCTONIANS", J. Differential Geometry 17 (1982) 185-232; all errors are definitely ours.

0. What this notebook does, and how it is organized

PART I (Sections 1-9) rebuilds, verifies and optimizes the algebraic engine of the original notebook: the flat 4+4 Minkowski metric, the real 8x8 Clifford generators tau, the 16x16 Dirac matrices T16, the so(4,4) generators SAB, the 256-element basis of the 16x16 matrix algebra, the 4x4 Dirac matrices, and the split-octonion structure constants obtained by Cartan triality.

PART II (Sections 10-14) reproduces the original physics: the 16-component wave function Psi16 of the "un-universe", its Lagrangian, the Euler-Lagrange equations, the change of variables to light-cone-like coordinates (z,t), the decoupling into four blocks of four equations, the Maple closed-form solutions, the bilinear invariants, and the M6 = 3 generations of Einstein-Rosen 2-plane bridges.

PART III (Sections 15-17) is new. It makes explicit the local flat 4+4 Minkowski coordinate system that exists at every point of the curved 4+4 spacetime, i.e. the frame field (vielbein) e , and then computes

- [a] the curved metric $g[\mu, \nu]$,
- [b] the flat Minkowski metric $\eta[a, b]$,
- [c] the frame field $e[\mu, a]$,
- [d] the bridge $g[\mu, \nu] == e[\mu, a] \eta[a, b] e[\nu, b]$,
- [e] the canonical spin connection $\omega[\mu, a, b]$ from the zero-torsion vielbein postulate $D[e[\nu, a], x[\mu]] - \Gamma[\rho, \mu, \nu] e[\rho, a] + \omega[\mu, a, b] e[\nu, b] == 0$,
- [f] the gauge-covariant derivative of the 16-component spinor $D[\psi, x[\mu]] + (1/8) \omega[\mu, a, b] \text{Commutator}[\gamma[a], \gamma[b]] . \psi$,
where $\gamma[a]$ are the 16x16 Dirac matrices T16 built in Section 5.

A vielbein here is simply an 8-dimensional vierbein, i.e. an 8-dimensional frame field.

PART IV (Sections 18-21) is also new. It invents three further ways of bridging curved spacetime indices to flat tangent-space indices, derives a new spin connection for each, and compares every one of them, component by component, to the canonical spin connection of Part III.

1. Session setup

We keep the original notebook's session policy: Simplify gets a time constraint of 1 second per sub-problem and FullSimplify gets 3 seconds, spelling warnings are switched off, and the dimension of the algebra is 8. These are the original tolerances and they are not changed here.

```
(* --- provenance banner,
as in the original notebook ----- *)
Print["CopyRight (C) 2022, Patrick L. Nash, under the General Public License."];
Print["Please cite this work, and this web page, if you use it."];
Print[
  "Refactored for claude-fable_Einstein-Rosen-2-Planes.nb; Wolfram Language ", $Version];

(* --- session
options: identical to the original ----- *)
SetOptions[Simplify, TimeConstraint -> 1];
SetOptions[FullSimplify, TimeConstraint -> 3];
{Off[General::spell], Off[General::spell1]};
Options[Simplify]
Options[FullSimplify]
```

```
(* --- the one global dimension
  constant ----- *)
ClearAll[DIM8];
DIM8 = 8;
(* M (mass), K and H (the Hubble-like constant)
  are the model's three scalar constants; the *)
(* original notebook protects them here so
  that no later cell can accidentally assign them. *)
Protect[DIM8, M, K, H];
{DIM8, Attributes[M], Attributes[K], Attributes[H]}
```

A small timing helper. Every expensive step in this notebook is wrapped in it so that a reader can see immediately where the time goes. It prints the wall-clock cost and returns the value unchanged, so it can be inserted anywhere without altering the mathematics.

```
ClearAll[cfTimed];
SetAttributes[cfTimed, HoldRest];
cfTimed[label_String, expr_] := Module[{t, r},
  {t, r} = AbsoluteTiming[expr];
  Print["  ",
    ToString[NumberForm[N[t], {6, 3}, ExponentFunction -> (Null &)], " s] ", label];
  r];
```

A reporting helper for the many algebraic identities that this notebook verifies. It reduces a whole table of assertions to a single True/False and prints a one-line verdict, which is far cheaper to read than the original notebook's pages of nested True lists.

```
ClearAll[cfAssert, $cfAssertLog];
$cfAssertLog = {};
cfAssert[label_String, value_] := Module[{flat, ok},
  flat = Union[Flatten[{value}]];
  ok = (flat === {True});
  AppendTo[$cfAssertLog, {label, ok}];
  Print[If[ok, " PASS ", " FAIL "], label,
    If[ok, "", " -> distinct values: " <> ToString[Short[flat, 3], InputForm]]];
  ok];

(* --- summary of every assertion made so
  far ----- *)
ClearAll[cfAssertSummary];
cfAssertSummary[] := Module[{n, bad},
  n = Length[$cfAssertLog];
  bad = Cases[$cfAssertLog, {l_, False} -> l];
  Print["Assertions run: ", n,
    " passed: ", n - Length[bad], " failed: ", Length[bad]];
  If[bad != {}, Print["FAILED: ", Column[bad]]];
  Grid[Prepend[$cfAssertLog, {"assertion", "ok"}], Frame -> All, Alignment -> Left];
```

2. Coordinates and the flat 4+4 Minkowski metric eta

Coordinates, following the original notebook (which took them from the author's earlier Eternal-DEFLATION-Inflation work):

```
{ x0 | x1 x2 x3 | x4 | x5 x6 x7 }
|   |   |   |
|   |   |   +-- superluminal deflating time directions (3)
|   |   +----- time
|   +----- ordinary 3-space
+----- hidden space
```

The flat metric eta4488 has signature (4,4): the first four directions are spacelike (+1) and the last four are timelike (-1). Throughout this notebook a capital Latin index A,B,... or a lower-case a,b,... is a FLAT tangent-space index and runs 0..7; a Greek index mu,nu,alpha,... is a CURVED spacetime index and also runs 0..7. Wolfram array positions are 1-based, so the flat index A sits at array position A+1. Every place where that shift matters is flagged.

```
ClearAll[X, x0, x1, x2, x3, x4, x5, x6, x7];
X = {x0, x1, x2, x3, x4, x5, x6, x7};
Protect[X, x0, x1, x2, x3, x4, x5, x6, x7];
X

(* --- the flat 4+
  4 Minkowski metric eta_{AB} ----- *)
ClearAll[eta4488, ID4, ID8, ID16, Zero4, Zero8, ZERO16];
ID4 = IdentityMatrix[4];
ID8 = IdentityMatrix[8];
ID16 = IdentityMatrix[16];
Zero4 = ConstantArray[0, {4, 4}];
Zero8 = ConstantArray[0, {8, 8}];
ZERO16 = ConstantArray[0, {16, 16}];
eta4488 = ArrayFlatten[{{ID4, 0}, {0, -ID4}}];
MatrixForm[eta4488]

(* --- eta is its own inverse,
  so raising and lowering flat indices is cheap ----- *)
cfAssert["eta4488 . eta4488 == ID8", eta4488 . eta4488 === ID8];
cfAssert["eta4488 is symmetric", eta4488 === Transpose[eta4488]];
cfAssert["signature of eta4488 is (4,4)",
  Sort[Diagonal[eta4488]] === Sort[{1, 1, 1, 1, -1, -1, -1, -1}]];
Diagonal[eta4488]
```

The Levi-Civita symbols of the original notebook. epsilon3 is used for the SO(3) structure constants and epsilon4 for the 4-dimensional duality operation that splits the antisymmetric 4x4 matrices into self-dual and anti-self-dual halves.

```

ClearAll[ε3, ε4];
ε3 = Array[Signature[{{##}}] &, {3, 3, 3}];
ε4 = Array[Signature[{{##}}] &, {4, 4, 4, 4}];
{Dimensions[ε3], Dimensions[ε4], ε4[[1, 2, 3, 4]]}

```

Positivity assumptions used by Simplify throughout. These are exactly the assumptions of the original notebook: constraintVars for the (z,t) chart and ssX for the (x0,x4) chart.

Note that la is a free scalar function inherited from the original notebook; it is never given a definition there and we do not invent one here.

```

ClearAll[sX0, ssX, constraintVars];
sX0 = And @@ Thread[X > 0];
ssX = H > 0 && sX0 && 6 H x0 > 0 && 2 la[x4] > 0 && Cot[6 H x0] > 0 && Sin[6 H x0] > 0;
constraintVars = x0 > 0 && x4 > 0 && z > 0 && t > 0;
{sX0, ssX, constraintVars}

(* --- the (x0,x4) ↔
   (z,t) chart change of the original notebook ----- *)
ClearAll[szt, sx0x4];
szt = Solve[6 H x0 == z && H x4 == t, {z, t}][[1]];
sx0x4 = Solve[6 H x0 == z && H x4 == t, {x0, x4}][[1]];
Protect[szt, sx0x4];
{szt, sx0x4}

```

3. SO(4): self-dual and anti-self-dual antisymmetric 4x4 matrices

The six-dimensional space of antisymmetric real 4x4 matrices splits, under the Hodge star built from epsilon4, into a self-dual 3-space and an anti-self-dual 3-space. The original notebook builds explicit bases for both from two elementary tensors:

$$Qa[h,p,q] = \text{Signature}[\{h,p,q,4\}] \quad (\text{the 3-dimensional epsilon})$$

$$Qb[h,p,q] = \text{KroneckerDelta}[p,4] \text{KroneckerDelta}[q,h]$$

$$- \text{KroneckerDelta}[p,h] \text{KroneckerDelta}[q,4]$$

with s4by4[h] = Qa - Qb self-dual and t4by4[h] = Qa + Qb anti-self-dual, h = 1,2,3.

These are the two commuting su(2) factors of so(4) = su(2) + su(2).

```

ClearAll[Qa, Qb, cfSelfDual, cfAntiSelfDual, s4by4, t4by4];
Qa[h_Integer, p_Integer, q_Integer] := Signature[{h, p, q, 4}];
Qb[h_Integer, p_Integer, q_Integer] := ID4[[p, 4]] ID4[[q, h]] - ID4[[p, h]] ID4[[q, 4]];
cfSelfDual[h_, p_, q_] := Qa[h, p, q] - Qb[h, p, q];
cfAntiSelfDual[h_, p_, q_] := Qa[h, p, q] + Qb[h, p, q];
Do[s4by4[h] = Table[cfSelfDual[h, p, q], {p, 4}, {q, 4}], {h, 1, 3}];
Do[t4by4[h] = Table[cfAntiSelfDual[h, p, q], {p, 4}, {q, 4}], {h, 1, 3}];
Protect[s4by4, t4by4];
Grid[{"h", "s4by4[h]", "t4by4[h]"},
  Sequence @@ Table[{h, MatrixForm[s4by4[h]], MatrixForm[t4by4[h]]}, {h, 3}],
  Frame → All]

(* --- duality checks,
exactly the two tests of the original notebook ----- *)
cfAssert["s4by4 is self-dual",
  Table[(1/2) Sum[ε4[[p, q, j1, j2]] s4by4[h][[j1, j2]], {j1, 4}, {j2, 4}] ==
    s4by4[h][[p, q]], {h, 3}, {p, 4}, {q, 4}]];
cfAssert["t4by4 is anti-self-dual",
  Table[(1/2) Sum[ε4[[p, q, j1, j2]] t4by4[h][[j1, j2]], {j1, 4}, {j2, 4}] ==
    -t4by4[h][[p, q]], {h, 3}, {p, 4}, {q, 4}]];
cfAssert["s4by4 and t4by4 are antisymmetric",
  Join[Table[s4by4[h] == -Transpose[s4by4[h]], {h, 3}],
    Table[t4by4[h] == -Transpose[t4by4[h]], {h, 3}]];
cfAssert["the two su(2) factors commute",
  Table[s4by4[j] . t4by4[k] == t4by4[k] . s4by4[j], {j, 3}, {k, 3}]];

(* --- the nine mixed products st[J,K],
used to complete the 4x4 basis ----- *)
ClearAll[st];
Do[st[J, K] = s4by4[J] . t4by4[K], {J, 1, 3}, {K, 1, 3}];
Protect[st];
cfAssert["st[J,K] is symmetric",
  Table[st[J, K] == Transpose[st[J, K]], {J, 3}, {K, 3}]];
Grid[Table[MatrixForm[st[J, K]], {J, 3}, {K, 3}], Frame → All]

```

4. The real 8x8 Clifford generators tau, their conjugates, and the spinor metric sigma

sigma is the $O(4,4)$ spinor metric. It is the 8x8 symmetric matrix that pairs the first four spinor components with the last four; it is its own inverse and it is traceless. In the language of the original notebook, both the type-1 and the type-2 split-octonion spinors carry this same sigma (the usual notational abuse), since $\text{sigma} == \text{Inverse}[\text{sigma}]$.

```

ClearAll[σ];
σ = ArrayFlatten[{{Zero4, ID4}, {ID4, Zero4}}];
cfAssert["sigma . sigma == ID8", σ . σ === ID8];
cfAssert["sigma is symmetric", σ === Transpose[σ]];
cfAssert["Tr[sigma] == 0", Tr[σ] === 0];
MatrixForm[σ]

```

Six 8x8 matrices are assembled from the SO(4) blocks. The first three use the self-dual blocks in a symmetric off-diagonal arrangement, the last three use the anti-self-dual blocks in an antisymmetric arrangement, and the anti-self-dual ones are taken in reverse order $h = 3, 2, 1$ so that the resulting tau matrices come out in the order fixed by the original notebook. That ordering matters: it is what makes $\text{tau}[7]$ equal the product $\text{tau}[1] \dots \text{tau}[6]$ and what makes sigma equal $\text{tau}[1].\text{tau}[2].\text{tau}[3]$.

A caution for readers of the original notebook. The variable is called `sixAntiSymmetric8by8` there, and we keep that name so the two notebooks can be read side by side, but the name is a misnomer for half of its contents: only the first three matrices are antisymmetric. The last three are SYMMETRIC, because transposing `ArrayFlatten[{{0,t},{-t,0}}]` with t antisymmetric gives the matrix back unchanged. That is not a defect. It is precisely what the signature demands, and the correct statement is verified below:

$$\text{Transpose}[\text{tau}[A]] == -\text{eta}4488[[A+1,A+1]] \text{tau}[A], \quad A = 1..7,$$

so $\text{tau}[1], \text{tau}[2], \text{tau}[3]$ (where $\text{eta} = +1$) are antisymmetric and $\text{tau}[4], \text{tau}[5], \text{tau}[6]$ (where $\text{eta} = -1$) are symmetric.

```

ClearAll[sixAntiSymmetric8by8];
sixAntiSymmetric8by8 = Join[
  Table[ArrayFlatten[{{0, s4by4[h]}, {s4by4[h], 0}}], {h, 1, 3}],
  Table[ArrayFlatten[{{0, t4by4[h]}, {-t4by4[h], 0}}], {h, 3, 1, -1}]];
cfAssert["blocks 1-3 (built from s4by4) are antisymmetric",
  Table[sixAntiSymmetric8by8[[k]] === -Transpose[sixAntiSymmetric8by8[[k]], {k, 1, 3}]];
cfAssert["blocks 4-6 (built from t4by4) are symmetric",
  Table[sixAntiSymmetric8by8[[k]] === Transpose[sixAntiSymmetric8by8[[k]], {k, 4, 6}]];
Length[sixAntiSymmetric8by8]

```

The 8x8 real Clifford generators. $\text{tau}[0]$ is the identity; $\text{tau}[1].. \text{tau}[6]$ are the six antisymmetric blocks just built; $\text{tau}[7]$ is their ordered product. The conjugate generator is $\text{taubar}[A] = \text{sigma} . \text{Transpose}[\text{sigma} . \text{tau}[A]]$, which is the sigma-adjoint of $\text{tau}[A]$.

Together they realize the split Clifford algebra of the 4+4 quadratic form in its 8x8 real "half-spinor" form:

$$(1/2) (\text{tau}[A] . \text{taubar}[B] + \text{tau}[B] . \text{taubar}[A]) == \text{eta}4488[[A+1,B+1]] \text{ID8} .$$

```

ClearAll[τ, τbar];
τ[0] = ID8;
Do[τ[h] = sixAntiSymmetric8by8[h], {h, 1, 6}];
τ[7] = τ[1] . τ[2] . τ[3] . τ[4] . τ[5] . τ[6];
Do[τbar[A] = σ . Transpose[σ . τ[A]], {A, 0, 7}];
MatrixForm[τ[7]]

(* --- the defining Clifford
relation ----- *)
cfAssert["(1/2) (tau[A].taubar[B] + tau[B].taubar[A]) == eta4488[[A+1,B+1]] ID8",
Table[(1/2) (τ[A] . τbar[B] + τ[B] . τbar[A]) ==
η4488[[A + 1, B + 1]] ID8, {A, 0, 7}, {B, 0, 7}]];

(* --- the identities of the original
notebook that fix the ordering conventions ----- *)
cfAssert["sigma == tau[1].tau[2].tau[3]", σ == τ[1] . τ[2] . τ[3]];
cfAssert["sigma == tau[4].tau[5].tau[6].tau[7]",
σ == τ[4] . τ[5] . τ[6] . τ[7]];
cfAssert["tau[1]...tau[7] == tau[0] == ID8",
τ[1] . τ[2] . τ[3] . τ[4] . τ[5] . τ[6] . τ[7] == ID8];
cfAssert["sigma . taubar[A] == Transpose[sigma . tau[A]]",
Table[σ . τbar[A] == Transpose[σ . τ[A]], {A, 0, 7}]];
cfAssert["-eta4488[[A+1,A+1]] tau[A] == Transpose[tau[A]] for A = 1..7",
Table[-η4488[[A + 1, A + 1]] τ[A] == Transpose[τ[A]], {A, 1, 7}]];

(* --- Omega =
sigma . tau[7] is the second invariant of the original notebook ----- *)
ClearAll[Ω];
Ω = σ . τ[7];
cfAssert["Omega == tau[4].tau[5].tau[6]",
Ω == τ[4] . τ[5] . τ[6]];
{Det[Ω], MatrixForm[Ω]}

```

5. The 16x16 Dirac matrices T16, the chirality operator, sigma16 and the so(4,4) generators

Following E. A. Lord's reduced Brauer-Weyl construction ("The Dirac spinor in six dimensions", Math. Proc. Camb. Phil. Soc. 64 (1968) 765-778), the 8x8 generators of Section 4 are promoted to 16x16 generators by the off-diagonal doubling

$$T16[A] = \text{ArrayFlatten}[\{ \{0, \text{taubar}[A]\}, \{\text{tau}[A], 0\} \}], \quad A = 0..7.$$

These are the Dirac matrices of this notebook. Everywhere below, $\gamma[a]$ means $T16[a]$. The doubling is exactly what realizes the 16-component spinor as the direct sum of a type-1 and a type-2 split-octonion spinor: the upper 8 components are the type-1 spinor and the lower 8 are the type-2 spinor, and each $T16[A]$ exchanges them.

They obey the full Clifford relation of the 4+4 quadratic form,

$$(1/2) (T16[A] \cdot T16[B] + T16[B] \cdot T16[A]) == \eta_{4488}[A+1, B+1] ID16,$$

equivalently $\text{Anticommutator}[\gamma[a], \gamma[b]] == 2 \eta[a, b] ID16$.

```
ClearAll[T16, σ16, covariantDiffMatrix];
Do[T16[A] = ArrayFlatten[{{0, τbar[A]}, {τ[A], 0}}, {A, 0, 7}];
(* T16[8] is the chirality operator: the ordered product of all eight generators. *)
T16[8] = Dot @@ Table[T16[A], {A, 0, 7}];
Dimensions[T16[0]]

(* --- the defining Clifford relation for
the 16x16 Dirac matrices ----- *)
(* Optimization:
the original wrapped this in FullSimplify. Every entry here is an exact *)
(* integer, so FullSimplify is a no-op that costs several seconds;
it is dropped. The *)
(* identity being verified is byte-for-
byte the one in the original notebook. *)
cfAssert["{T16[A], T16[B]}/2 == eta4488[[A+1,B+1]] ID16",
Table[(1/2) (T16[A] . T16[B] + T16[B] . T16[A]) == η4488[[A + 1, B + 1]] ID16,
{A, 0, 7}, {B, 0, 7}]];

(* --- sigma16 is the 16x16 spinor metric;
it is the product of the four spacelike gammas -- *)
σ16 = T16[0] . T16[1] . T16[2] . T16[3];
cfAssert["sigma16 == ArrayFlatten[{{-sigma,0},{0,sigma}}]",
σ16 == ArrayFlatten[{{-σ, 0}, {0, σ}}]];
cfAssert["sigma16 . T16[A] is antisymmetric for A = 0..7",
Table[σ16 . T16[A] == -Transpose[σ16 . T16[A]], {A, 0, 7}]];
cfAssert["T16[8] == sigma16 . T16[4].T16[5].T16[6].T16[7]",
T16[8] == σ16 . T16[4] . T16[5] . T16[6] . T16[7]];
MatrixForm[σ16]

(* --- the matrix that the original
notebook uses to build covariant differences ----- *)
covariantDiffMatrix = T16[5] . T16[6] . T16[7];
cfAssert["sigma16 . covariantDiffMatrix is symmetric",
σ16 . covariantDiffMatrix == Transpose[σ16 . covariantDiffMatrix]];
MatrixForm[covariantDiffMatrix]
```

Chirality projectors. $T16[8]$ squares to the identity, so $(ID16 \mp T16[8])/2$ are complementary orthogonal projectors that split the 16-component spinor into its two 8-component chiral halves.

```

ClearAll[PL, PR];
PL = (1/2) (ID16 - T16[8]);
PR = (1/2) (ID16 + T16[8]);
cfAssert["PL + PR == ID16", PL + PR === ID16];
cfAssert["PL is idempotent", PL . PL === PL];
cfAssert["PR is idempotent", PR . PR === PR];
cfAssert["PL . PR == PR . PL == 0", {PL . PR === ZERO16, PR . PL === ZERO16}];
{MatrixForm[PL], MatrixForm[PR]}

```

The so(4,4) Lorentz generators in the 16-dimensional spinor representation,

$$SAB[[A+1,B+1]] = S^{AB} = (1/4) \text{Commutator}[T16[A], T16[B]] .$$

These are the objects that appear in the spinor covariant derivative of Part III: the combination $(1/8) \omega_{\mu,a,b} \text{Commutator}[\gamma_a, \gamma_b]$ is exactly $(1/2) \omega_{\mu,a,b} S^{ab}$.

```

ClearAll[SAB, sAB];
SAB = Table[(1/4) (T16[A1] . T16[B1] - T16[B1] . T16[A1]), {A1, 0, 7}, {B1, 0, 7}];
(* the same objects addressed by flat index rather than array position,
as in the original *)
Do[sAB[A1][B1] = SAB[[A1 + 1, B1 + 1]], {A1, 0, 7}, {B1, 0, 7}];
cfAssert["SAB is antisymmetric in its two flat indices",
Table[SAB[[A1, B1]] === -SAB[[B1, A1]], {A1, 1, 8}, {B1, 1, 8}]];
(* sigma16 . SAB is ANTIsymmetric,
not symmetric. Proof: sigma16 is symmetric and squares to *)
(* ID16, and sigma16 . T16[A] is antisymmetric,
so Transpose[T16[A]] == -sigma16.T16[A].sigma16. *)
(* Substituting that twice into
Transpose[sigma16 . T16[A] . T16[B]] gives sigma16.T16[B].T16[A], *)
(* which flips the sign of the commutator. The
original notebook records both the correct claim *)
(* and, in an exploratory cell, the opposite one;
only the antisymmetric statement is true. *)
cfAssert["sigma16 . SAB is antisymmetric",
Table[sigma16 . SAB[[A1, B1]] === -Transpose[sigma16 . SAB[[A1, B1]], {A1, 1, 8}, {B1, 1, 8}]];
cfAssert["sigma16 is symmetric and sigma16 . sigma16 == ID16",
{sigma16 === Transpose[sigma16], sigma16 . sigma16 === ID16}];
Dimensions[SAB]

```

```
(* --- the so(4,4) commutation relations,
over the 28 independent index pairs ----- *)
(* [S_AB, S_CD] ==
  -( eta_AC S_BD - eta_AD S_BC - eta_BC S_AD + eta_BD S_AC )      *)
cfTimed["so(4,4) commutators over 28 x 28 index pairs",
cfAssert["[SAB,SAB] closes on the so(4,4) algebra",
Flatten @ Table[
  SAB[[A1, B1]] . SAB[[A2, B2]] - SAB[[A2, B2]] . SAB[[A1, B1]] ===
  -(eta4488[[A1, A2]] SAB[[B1, B2]] - eta4488[[A1, B2]] SAB[[B1, A2]]
    - eta4488[[B1, A2]] SAB[[A1, B2]] + eta4488[[B1, B2]] SAB[[A1, A2]]),
  {A1, 1, 7}, {B1, A1 + 1, 8}, {A2, 1, 7}, {B2, A2 + 1, 8}]]];

(* --- S_AB rotates the gammas as a
vector: [S_AB, gamma_C] == -eta_CA gamma_B + eta_CB gamma_A *)
cfTimed["vector transformation law of the gammas",
cfAssert["[SAB, T16] reproduces the vector representation",
Flatten @ Table[
  SAB[[A1, B1]] . T16[B2 - 1] - T16[B2 - 1] . SAB[[A1, B1]] ===
  -eta4488[[B2, A1]] T16[B1 - 1] + eta4488[[B2, B1]] T16[A1 - 1],
  {A1, 1, 8}, {B1, 1, 8}, {B2, 1, 8}]]];
```

6. A complete basis of the 16x16 matrix algebra

The Clifford algebra generated by eight 16x16 gammas has $2^8 = 256 = 16 \times 16$ independent elements, namely the ordered products of k distinct generators for $k = 1..8$ together with the identity. The original notebook builds these with eight nested Do loops and AppendTo. We build the same list, in exactly the same order, with Subsets, which enumerates index tuples in lexicographic order and is both shorter and quadratically faster. The order matters because the original refers to particular positions in it (93 and 255), and those references are re-verified below.

```
ClearAll[cfCliffordWord, base16];
(* cfCliffordWord[{j1,...,jk}] is the ordered product T16[j1]. ... .T16[jk] *)
cfCliffordWord[idx_List] := Dot @@ (T16 /@ idx);
base16 = cfTimed["build the 256-element basis of the 16x16 algebra",
Join[
  Sequence @@
    Table[{cfCliffordWord[#], #} & /@ Subsets[Range[0, 7], {k}], {k, 1, 8}],
  {{ID16, {Range[0, 7]}}}]]];
{Length[base16], Length[Union[base16[[All, 1]]]}

(* --- the two positional references made
by the original notebook ----- *)
cfAssert["base16[[93]] is sigma16", sigma16 === base16[[93, 1]];
cfAssert["base16[[255]] is the chirality operator T16[8]", T16[8] === base16[[255, 1]];
cfAssert["base16[[256]] is the identity", ID16 === base16[[256, 1]];
{base16[[93, 2]], base16[[255, 2]], base16[[256, 2]]}
```

```

(* --- every basis element squares to +ID16 or -ID16,
so Tr[M.M]/16 is +1 or -1 ----- *)
cfAssert["Tr[M.M]/16 is +1 or -1 for every basis element",
  MemberQ[{1, -1}, #] & /@ Union[Flatten[Tr[#, #/16] & /@ base16[All, 1]]]];
Union[Flatten[Tr[#, #/16] & /@ base16[All, 1]]]

(* --- split the basis by the sign of Tr[M.M];
the original records 136 and 120 ----- *)
ClearAll[positiveTrMM, negativeTrMM];
positiveTrMM = Select[base16, Tr[#[[1]] . #[[1]]] > 0 & → "Index"];
negativeTrMM = Select[base16, Tr[#[[1]] . #[[1]]] < 0 & → "Index"];
cfAssert["136 basis elements have Tr[M.M] > 0", Length[positiveTrMM] === 136];
cfAssert["120 basis elements have Tr[M.M] < 0", Length[negativeTrMM] === 120];
{Length[positiveTrMM], Length[negativeTrMM]}

(* --- and the same split is the
symmetric / antisymmetric split: 16*17/2 and 16*15/2 ----- *)
ClearAll[symm16, antisymm16];
symm16 = Select[Range[Length[base16]], base16[[#, 1]] === Transpose[base16[[#, 1]]] &];
antisymm16 =
  Select[Range[Length[base16]], base16[[#, 1]] === -Transpose[base16[[#, 1]]] &];
cfAssert["symmetric basis elements number 16*17/2 = 136", Length[symm16] === 136];
cfAssert["antisymmetric basis elements number 16*15/2 = 120",
  Length[antisymm16] === 120];
cfAssert["Tr[M.M] > 0 exactly when M is symmetric",
  Sort[symm16] === Sort[positiveTrMM]];
cfAssert["Tr[M.M] < 0 exactly when M is antisymmetric",
  Sort[antisymm16] === Sort[negativeTrMM]];
{Length[symm16], Length[antisymm16], 16 (16 + 1) / 2, 16 (16 - 1) / 2}

(* --- the original also removes sign-
duplicates to get a projective basis ----- *)
ClearAll[dups, BASE16, use16];
dups = cfTimed["find sign-duplicate basis elements",
  Select[base16, MemberQ[base16[All, 1], -#[[1]]] &]];
BASE16 = use16 = Complement[base16, dups];
{Length[dups], Length[use16]}

```

7. A complete basis of the 8x8 matrix algebra, and the induced flat metric

The same construction one level down. The 8x8 algebra generated by $\tau[1].. \tau[7]$ has $7 + 21 + 35 = 63$ independent products plus the identity, i.e. $64 = 8 \times 8$ elements. The trace form $(1/8) \text{Tr}[M.M]$ on this basis is the diagonal metric η_{64} ; its trace counts the signature.

```

ClearAll[bas64, η64];
bas64 = Join[
  Sequence @@ Table[{Dot @@ (τ /@ #), #} & /@ Subsets[Range[1, 7], {k}], {k, 1, 3}],
  {{ID8, {64}}}]];
cfAssert["the 8x8 basis has 64 elements", Length[bas64] === 64];
cfAssert["the 64 elements are distinct", Length[Union[bas64[[All, 1]]] === 64];
η64 = DiagonalMatrix[Table[(1/8) Tr[bas64[[A, 1]].bas64[[A, 1]], {A, 64}]];
{Length[bas64], Tr[η64], Union[Diagonal[η64]]}

(* --- the trace form is
orthogonal: (1/8) Tr[M_A . M_B] is diagonal ----- *)
cfTimed["orthogonality of the 64-element trace form",
cfAssert["(1/8) Tr[bas64[A].bas64[B]] == eta64[[A,B]]",
Flatten @ Table[(1/8) Tr[bas64[[A, 1]].bas64[[B, 1]] === η64[[A, B],
{A, 1, 64}, {B, 1, 64}]]];

(* --- symmetric / antisymmetric split of the 8x8 basis: 8*9/2 =
36 and 8*7/2 = 28 ----- *)
ClearAll[symm8, anti8];
symm8 = Select[Range[64], bas64[[#, 1]] === Transpose[bas64[[#, 1]] &];
anti8 = Select[Range[64], bas64[[#, 1]] === -Transpose[bas64[[#, 1]] &];
cfAssert["symmetric 8x8 basis elements number 36", Length[symm8] === 36];
cfAssert["antisymmetric 8x8 basis elements number 28", Length[anti8] === 28];
{Length[symm8], Length[anti8]}

```

8. The 4x4 Dirac matrices, and a complete basis of the 4x4 matrix algebra

The original notebook also carries the ordinary four-dimensional Dirac algebra, because the (x1,x2,x3,x4) block of the 4+4 metric is an ordinary Minkowski space of signature (+,+,-,-). Its 4x4 metric is the sub-block $g_{44} = \eta_{4488}[[2;;5, 2;;5]] = \text{diag}(1,1,1,-1)$.

The four gammas are built from the same self-dual and anti-self-dual SO(4) blocks of Section 3. With $g_3 = \text{diag}(1,1,-1)$ and $Gr[h,k] = -g_3[[h,h]] g_3[[k,k]] t_4by4[h] . s_4by4[k]$, the original takes

```
gamma = { Gr[1,1], Gr[1,2], Gr[1,3], t4by4[2] }, gamma[5] = -gamma[1].gamma[2].gamma[3].gamma[4]
```

and the four of them satisfy the four-dimensional Clifford relation

$\text{Anticommutator}[\text{gamma}[h], \text{gamma}[k]] = 2 g_{44}[[h,k]] \text{ID}_4$. The associated Lorentz generators are $S_{44} = -(1/4) \text{Commutator}[\text{gamma}[h], \text{gamma}[k]]$.

```

ClearAll[g44, g3, Gr, γ4];
g44 = η4488[[2 ;; 5, 2 ;; 5]];
g3 = DiagonalMatrix[{1, 1, -1}];
Do[Gr[h, k] = t4by4[h] . s4by4[k] (-g3[[h, h] g3[[k, k]]), {h, 1, 3}, {k, 1, 3}];
γ4 = {Gr[1, 1], Gr[1, 2], Gr[1, 3], t4by4[2]};
AppendTo[γ4, -γ4[[1]] . γ4[[2]] . γ4[[3]] . γ4[[4]]];
cfAssert["g44 == diag(1,1,1,-1)", g44 === DiagonalMatrix[{1, 1, 1, -1}]];
cfAssert["gamma4[[5]] == -t4by4[3]", γ4[[5]] === -t4by4[3]];
{MatrixForm[g44], MatrixForm /@ γ4}

(* --- the four-dimensional Clifford relation,
and the fifth gamma ----- *)
cfAssert["{gamma4[h], gamma4[k]} == 2 g44[[h,k]] ID4 for h,k = 1..4",
Table[γ4[[h]] . γ4[[k]] + γ4[[k]] . γ4[[h]] === 2 g44[[h, k]] ID4,
{h, 1, 4}, {k, 1, 4}]];
cfAssert["{gamma4[h], gamma4[k]} == 2 eta4488[[1+h,1+k]] ID4 for h,k = 1..5",
Table[γ4[[h]] . γ4[[k]] + γ4[[k]] . γ4[[h]] ===
2 η4488[[1 + h, 1 + k]] ID4, {h, 1, 5}, {k, 1, 5}]];

(* --- the 4x4 charge-conjugation matrix epsilon,
and the Lorentz generators S44 ----- *)
ClearAll[ε4, S44, η44];
ε4 = t4by4[2];
η44 = DiagonalMatrix[{1, 1, 1, -1}];
S44 = -(1/4) Table[γ4[[h]] . γ4[[k]] - γ4[[k]] . γ4[[h]], {h, 1, 4}, {k, 1, 4}];
cfAssert["epsilon4 is antisymmetric", Transpose[ε4] === -ε4];
(* epsilon4 is the charge-conjugation matrix of the four-
dimensional algebra. It is symmetric *)
(* against the four vector gammas and
ANTIsymmetric against the chirality gamma gamma4[[5]]. *)
(* The original notebook tests A =
1..5 in one exploratory cell and therefore sees a mixed *)
(* result; the two correct statements are
separated here. *)
cfAssert["epsilon4 . gamma4[A] is symmetric for A = 1..4",
Table[ε4 . γ4[[A]] === Transpose[ε4 . γ4[[A]]], {A, 1, 4}]];
cfAssert["epsilon4 . gamma4[5] is ANTIsymmetric",
ε4 . γ4[[5]] === -Transpose[ε4 . γ4[[5]]]];
cfAssert["epsilon4 . epsilon4 == -ID4", ε4 . ε4 === -ID4];
cfAssert["epsilon4 . S44 is symmetric",
Table[ε4 . S44[[A, B]] === Transpose[ε4 . S44[[A, B]]],
{A, 1, 3}, {B, A + 1, 4}]];
Dimensions[S44]

```

```

(* --- the so(3,1) commutation relations,
and the vector law for the 4x4 gammas ----- *)
cfAssert["[S44,S44] closes on the so(3,1) algebra",
Flatten @ Table[
  S44[[A1, B1]] . S44[[A2, B2]] - S44[[A2, B2]] . S44[[A1, B1]] ==
   $\eta_{44}[[A1, A2]] S44[[B1, B2]] - \eta_{44}[[A1, B2]] S44[[B1, A2]]$ 
  -  $\eta_{44}[[B1, A2]] S44[[A1, B2]] + \eta_{44}[[B1, B2]] S44[[A1, A2]]$ ,
  {A1, 1, 3}, {B1, A1 + 1, 4}, {A2, 1, 3}, {B2, A2 + 1, 4}]]];
cfAssert["[S44, gamma4] reproduces the vector representation",
Flatten @ Table[
  S44[[A1, B1]] .  $\gamma_4[[B2]]$  -  $\gamma_4[[B2]]$  . S44[[A1, B1]] ==
   $\eta_{44}[[B2, A1]] \gamma_4[[B1]] - \eta_{44}[[B2, B1]] \gamma_4[[A1]]$ ,
  {A1, 1, 4}, {B1, 1, 4}, {B2, 1, 4}]]];

(* --- a complete 16-
element basis of the 4x4 matrix algebra ----- *)
(* Built from the 2x2 Pauli blocks exactly as the original does,
with *)
(* sigma22 =
{ ID2, sigma1, I sigma2, sigma3 } (the original's If[##=2, I, 1]) *)
(* sigma22bar =
{-ID2, sigma1, I sigma2, sigma3 } *)
(* Every entry is real,
because I PauliMatrix[2] == {{0,1},{-1,0}}. *)
(* and xxx[j] =
ArrayFlatten[{{0, sigma22bar[j]}, {sigma22[j], 0}}]. *)
ClearAll[ $\sigma_{22}$ ,  $\sigma_{22bar}$ , xxx, base4by4,  $\eta_{2244}$ ];
 $\sigma_{22}$  = Join[{IdentityMatrix[2]}, Table[If[h == 2, I, 1] PauliMatrix[h], {h, 1, 3}]];
 $\sigma_{22bar}$  = Join[{-IdentityMatrix[2]}, Table[If[h == 2, I, 1] PauliMatrix[h], {h, 1, 3}]];
 $\eta_{2244}$  = DiagonalMatrix[{-1, 1, -1, 1}];
cfAssert[
"(1/2) (sigma22[A] sigma22bar[B] + sigma22[B] sigma22bar[A]) == eta2244[[A,B]] ID2",
Table[(1/2) ( $\sigma_{22}[[A]]$  .  $\sigma_{22bar}[[B]]$  +  $\sigma_{22}[[B]]$  .  $\sigma_{22bar}[[A]]$ ) ==
 $\eta_{2244}[[A, B]]$  IdentityMatrix[2], {A, 1, 4}, {B, 1, 4}]];
Do[xxx[j] = ArrayFlatten[{{0,  $\sigma_{22bar}[[j]]$ }, { $\sigma_{22}[[j]]$ , 0}}], {j, 1, 4}];
base4by4 = Join[{{ID4, {0}}},

Sequence @@ Table[{Dot @@ (xxx /@ #), #} & /@ Subsets[Range[1, 4], {k}], {k, 1, 4}]];
cfAssert["the 4x4 basis has 16 elements", Length[base4by4] == 16];
cfAssert["the 16 elements are linearly independent",
MatrixRank[Flatten /@ base4by4[[All, 1]]] == 16];
{Length[base4by4], MatrixRank[Flatten /@ base4by4[[All, 1]]]}

```

```
(* --- and the 1 + 4 + 6 + 4 +
  1 symmetry pattern of the 4x4 basis ----- *)
cfAssert["the 4x4 basis splits 10 symmetric + 6 antisymmetric",
  {Length[Select[base4by4, #[[1]] === Transpose[#[[1]] &]] === 10,
   Length[Select[base4by4, #[[1]] === -Transpose[#[[1]] &]] === 6}];
{Plus @@ {1, 4, 6, 4, 1},
 Length[Select[base4by4, #[[1]] === Transpose[#[[1]] &]]],
 Length[Select[base4by4, #[[1]] === -Transpose[#[[1]] &]]]}
```

9. Cartan triality: the constant bridge between the vector and spinor realizations, and the split-octonion structure constants

The split octonion algebra $O(4,4)$ carries three inequivalent-looking but equivalent 8-component representations of $\text{Spin}(4,4)$: the vector, the type-1 spinor and the type-2 spinor. That is Cartan's triality. The original notebook realizes the vector-to-spinor bridge concretely by picking one distinguished isotropic direction.

Take the eigensystem of the spinor metric σ . Its eigenvectors, rescaled by $1/\sqrt{2}$, form the matrix uEig . Rows 1..4 have eigenvalue -1 and rows 5..8 have eigenvalue +1. The original selects row $\text{hUSE} = 8$ and calls it unit ; it is a unit-norm spinor, $\text{Transpose}[\text{unit}].\sigma.\text{unit} == 1$.

```
ClearAll[evalues, evecs, uEig, hUSE, unit];
{evalues, evecs} = Eigensystem[σ];
uEig = ExpandAll[(1/Sqrt[2]) evecs];
hUSE = 8;
unit = uEig[[hUSE]];
Protect[hUSE, unit];
cfAssert["each uEig row is sigma-normalized to +/-1",
  MemberQ[{1, -1}, #] & /@ Table[uEig[[h]].σ.uEig[[h]], {h, 1, 8}]];
cfAssert["unit is sigma-normalized to +1", unit.σ.unit === 1];
{evalues, MatrixForm[uEig], unit}
```

The triality bridge itself. Define the 8x8 matrix

$$\text{triVecToSpin}[[A+1, a+1]] = (\text{unit}.\sigma.\text{taubar}[A])[[a+1]],$$

which carries a FLAT VECTOR index A (row) and a FLAT TYPE-1 SPINOR index a (column). This is the object written $E^{\{A\}}_a$, equivalently $F_a^{\{A\}}$, in the original notebook. Its inverse triSpinToVec carries the indices the other way round, and the original gives a closed form for it that we verify rather than assume.

```

ClearAll[triVecToSpin, triSpinToVec];
triVecToSpin = Table[unit .  $\sigma$  .  $\tau$ bar[A], {A, 0, 7}];
triSpinToVec = Inverse[triVecToSpin];
cfAssert["triVecToSpin . triSpinToVec == ID8", triVecToSpin . triSpinToVec === ID8];
cfAssert["triSpinToVec . triVecToSpin == ID8", triSpinToVec . triVecToSpin === ID8];
(* the original's closed form for the inverse bridge *)
cfAssert[
  "triSpinToVec == Transpose[ eta_AA tau[A].unit ] (the original's closed form)",
  triSpinToVec === Transpose[Table[ $\eta$ 4488[A + 1, A + 1] ( $\tau$ [A] . unit), {A, 0, 7}]]];
MatrixForm[triVecToSpin]

```

What flat tangent metric does the triality bridge induce? If a vector v with flat components v^A is carried to the spinor components $w^a = v^A \text{triVecToSpin}[[A,a]]$, then the flat length $v.\text{eta}.v$ becomes $w . \text{etaTri} . w$ with

$$\text{etaTri} = \text{Inverse}[\text{triVecToSpin}] . \text{eta4488} . \text{Inverse}[\text{Transpose}[\text{triVecToSpin}]] .$$

We compute it and find that it is EXACTLY the split-octonion spinor metric σ . That is the precise sense in which the type-1 spinor space is an isometric copy of the 4+4 vector space, and it is what makes the third new bridge of Section 18 possible.

```

ClearAll[ $\eta$ Tri];
 $\eta$ Tri = Simplify[triSpinToVec .  $\eta$ 4488 . Transpose[triSpinToVec]];
cfAssert["etaTri is symmetric",  $\eta$ Tri === Transpose[ $\eta$ Tri]];
cfAssert["etaTri has signature (4,4)",
  Sort[Eigenvalues[ $\eta$ Tri]] === Sort[{1, 1, 1, 1, -1, -1, -1, -1}]];
(* w_a = v^A triVecToSpin[[A,a]],
so v.eta.v == w.etaTri.w means eta == P . etaTri . Transpose[P] . *)
cfAssert["triVecToSpin transports etaTri back to eta4488",
  Simplify[triVecToSpin .  $\eta$ Tri . Transpose[triVecToSpin] -  $\eta$ 4488] === Zero8];
cfAssert["THE TRIALITY METRIC IS EXACTLY THE SPINOR METRIC: etaTri == sigma",
   $\eta$ Tri ===  $\sigma$ ];
MatrixForm[ $\eta$ Tri]

```

The split-octonion multiplication constants. With the triality bridge in hand, the product of two basis units is read off from the Clifford generators:

$$\begin{aligned} \text{mabc}[[a,b,c]] &= \text{Sum over } A \text{ of } \text{triVecToSpin}[[A,a]] * (\text{eta_AA tau}[A-1])[[c,b]] \\ \text{mABC}[[A,B,C]] &= \text{Sum over } b,c \text{ of } \text{triVecToSpin}[[C,c]] (\text{eta_AA tau}[A-1])[[c,b]] \text{triSpinToVec}[[b,B]] \end{aligned}$$

mabc is the multiplication table in the type-1 spinor basis; mABC is the same table in the vector basis, which is the one usually displayed as "the split octonion multiplication table".

```

ClearAll[τA, mabc, mABC];
τA = Table[η4488[A, A] τ[A - 1], {A, 1, 8}];
mabc = cFTimed["split-octonion structure constants in the spinor basis",
  Table[Sum[triVecToSpin[A1, a1] τA[A1][c1, b1], {A1, 1, 8}],
    {a1, 1, 8}, {b1, 1, 8}, {c1, 1, 8}]];
mABC = cFTimed["split-octonion structure constants in the vector basis",
  Table[Sum[triVecToSpin[C1, c1] τA[A1][c1, b1] triSpinToVec[b1, B1],
    {b1, 1, 8}, {c1, 1, 8}, {A1, 1, 8}, {B1, 1, 8}, {C1, 1, 8}]];
{Dimensions[mabc], Dimensions[mABC]}

(* --- the vector basis units,
and the multiplication table as the original displays it ---- *)
ClearAll[EA, gridSplitOctonion];
EA = Array[eA, 8];
gridSplitOctonion = Grid[
  Prepend[
    Table[
      Flatten[{EA[A1], Table[Sum[mABC[A1, B1, C1] EA[C1], {C1, 1, 8}], {B1, 1, 8}]}],
      {A1, 1, 8}],
    Flatten[{"A/B", EA}]],
  Frame → All];
gridSplitOctonion

(* --- eA[1] is the unit of the algebra: eA[1] x =
x eA[1] = x ----- *)
cfAssert["eA[1] is a two-sided identity for the vector-basis product",
  Join[
    Flatten @ Table[Sum[mABC[1, B1, C1] ID8[C1], {C1, 8}] == ID8[B1], {B1, 1, 8}],
    Flatten @ Table[Sum[mABC[A1, 1, C1] ID8[C1], {C1, 8}] == ID8[A1], {A1, 1, 8}]]];
Table[Sum[mABC[1, B1, C1] EA[C1], {C1, 8}], {B1, 8}]

(* --- the norm is multiplicative on basis units: m^C_AB eta_CD m^D_AB =
eta_AA eta_BB ----- *)
cfAssert["<eA, eB> composition is compatible with eta (basis-unit norm form)",
  Flatten @ Table[
    Sum[mABC[A1, B1, C1] η4488[C1, D1] mABC[A1, B1, D1], {C1, 8}, {D1, 8}] ==
    η4488[A1, A1] η4488[B1, B1],
    {A1, 1, 8}, {B1, 1, 8}]];

```

```
(* --- the imaginary structure constants
are totally antisymmetric ----- *)
(* Lower the last index with eta and drop the real direction A=
1 (the unit eA[1]). *)
ClearAll[mLower, mImTotallyAntisymmetric];
mLower = Table[Sum[mABC[[A1, B1, D1]]  $\eta$ 4488[[D1, C1]], {D1, 8}],
{A1, 1, 8}, {B1, 1, 8}, {C1, 1, 8}];
mImTotallyAntisymmetric = Flatten @ Table[
{mLower[[A1, B1, C1]] === -mLower[[B1, A1, C1]],
mLower[[A1, B1, C1]] === -mLower[[A1, C1, B1]]},
{A1, 2, 8}, {B1, 2, 8}, {C1, 2, 8}];
cfAssert["m_{ABC} restricted to imaginary directions is totally antisymmetric",
mImTotallyAntisymmetric];
Total[Abs[Flatten[mLower[[2 ;; 8, 2 ;; 8, 2 ;; 8]]]]]
```

PART II -- The wave function of the un-universe, its field equations, and the 3 generations

10. Housekeeping: where this notebook lives, and the two helper packages

The original notebook calls NotebookFileName[] directly. That works in the Wolfram front end but returns \$Failed in a headless kernel, which would stop the whole notebook. We resolve the location defensively instead, and fall back on the current working directory. Two helper packages written by the author are loaded if they can be found:

```
ConvertMapleToMathematicaV2.wl  parses the Maple solution strings of Section 12
EtoExp.wl                       rewrites Power[E, ...] as Exp[...]
```

If a package cannot be found we say exactly which file is missing and where we looked, and we do NOT substitute a home-made replacement for it.

```

ClearAll[cfNotebookFile, cfDir, cfName, cfHeader, cfResolveFile];
cfNotebookFile = Quiet[NotebookFileName[]];
cfDir = If[StringQ[cfNotebookFile], DirectoryName[cfNotebookFile], Directory[]];
cfName = If[StringQ[cfNotebookFile], FileNameBaseName[cfNotebookFile],
  "claude-fable_Einstein-Rosen-2-Planes"];
cfHeader = cfName <> "-";
{cfDir, cfName, cfHeader}

(* --- look for a file next to the notebook,
in the working directory, or one level down --- *)
cfResolveFile[name_String] := Module[{cands, hit},
  cands = DeleteDuplicates[{
    FileNameJoin[{cfDir, name}],
    FileNameJoin[{cfDir, "Pre-Universe_14SEP26-77", name}],
    FileNameJoin[{Directory[], name}],
    FileNameJoin[{Directory[], "Pre-Universe_14SEP26-77", name}],
    FileNameJoin[{ParentDirectory[Directory[]], name}]]];
  hit = SelectFirst[cands, FileExistsQ[#] &, $Failed];
  If[hit === $Failed,
    Print[" MISSING FILE ", name];
    Print["          searched: ", Column[cands]];
    Print[
      "          Put the file in one of those places and re-evaluate this cell."];];
  hit];
{cfResolveFile["ConvertMapleToMathematicaV2.wl"], cfResolveFile["EtoExp.wl"]}

(* --- load
them ----- *)
ClearAll[cfMaplePath, cfEtoExpPath];
cfMaplePath = cfResolveFile["ConvertMapleToMathematicaV2.wl"];
cfEtoExpPath = cfResolveFile["EtoExp.wl"];
If[StringQ[cfMaplePath], Get[cfMaplePath],
  Print[" ConvertMapleToMathematicaV2 not loaded; Section 12 will report it again."]];
If[StringQ[cfEtoExpPath], Get[cfEtoExpPath],
  Print[
    " EtoExp not loaded; the Exp-rewriting cells of Section 12 will report it again."]];
{ValueQ[cfMaplePath], ValueQ[cfEtoExpPath]}

```

11. The wave function of the un-universe, and its Lagrangian

Psi16 is the 16-component spinor that the original notebook calls the wave function of the un-universe. Its components are sixteen scalar functions $f_{16}[0..15]$ of the two coordinates that the model actually depends on, the hidden-space coordinate x_0 and the time coordinate x_4 . The first eight components are the type-1 split-octonion spinor and the last eight are the type-2 spinor, so Psi16 is literally the direct sum of the two.

The original notebook also prepares two substitution rules that rewrite the f16 in terms of new unknowns in the light-cone-like chart $z = 6 H x_0$, $t = H x_4$:

```
sfpsi16Aa f16[k] -> ( Z[k][6 H #1, H #2] & )
snewfpsi16Aa f16[k] -> ( nZ[k][6 H #1, H #2] / Sqrt[Sin[6 H #1]] & )
```

Only the first is used downstream; the second is the author's alternative normalization and is recorded here unchanged. In the original the second rule's $1/\text{Sqrt}[\text{Sin}[\dots]]$ factor is written inside the string as a typeset `FractionBox` escape; we write it in plain input form, which parses to exactly the same expression.

```
ClearAll[ψ16, ψ16upper, ψ16lower, f16];
ψ16 = Table[f16[k][x0, x4], {k, 0, 15}];
ψ16upper = Take[ψ16, 8]; (* type-1 split-octonion spinor *)
ψ16lower = Take[ψ16, -8]; (* type-2 split-octonion spinor *)
cfAssert["Psi16 is the direct sum of the type-1 and type-2 spinors",
  ψ16 === Join[ψ16upper, ψ16lower]];
ψ16

(* --- the two re-
parameterizations of the component functions ----- *)
ClearAll[sfψ16Aa, snewfψ16Aa, sreplaceZ, sreplacenZ, subsDefects];
sfψ16Aa =
Table[
  f16[k] → ToExpression["(Z[" <> ToString[k] <> "][6*H*#1,H*#2])&"], {k, 0, 15}];
snewfψ16Aa =
Table[f16[k] → ToExpression[
  "(nZ[" <> ToString[k] <> "][6*H*#1,H*#2]/Sqrt[Sin[6*H*#1]])&"], {k, 0, 15}];
sreplaceZ = Table[Z[k] → ToExpression["Z" <> ToString[k]], {k, 0, 15}];
sreplacenZ = Table[nZ[k] → ToExpression["nZ" <> ToString[k]], {k, 0, 15}];
subsDefects = {}; (* the original's hook for patching defective components; empty *)
{sfψ16Aa[[1]], snewfψ16Aa[[1]], Length[sreplaceZ], subsDefects}
```

The Lagrangian. This is the Dirac Lagrangian of the pre-universe, written with the 16x16 Dirac matrices of Section 5 and the 16x16 spinor metric σ_{16} :

$$\begin{aligned} \mathcal{L} = & (1/H) \Psi_{16}^T \cdot \sigma_{16} \cdot \text{Sum over } \alpha \text{ of } T_{16}[\alpha-1] \cdot D[\Psi_{16}, x_\alpha] \\ & + 2 (M/H) \Psi_{16}^T \cdot \sigma_{16} \cdot \Psi_{16}. \end{aligned}$$

Because Ψ_{16} depends only on x_0 and x_4 , only the $\alpha = 1$ and $\alpha = 5$ terms survive.

There is an exact structural identity behind this Lagrangian that is worth stating, because the original notebook probes it in a scratch cell with mismatched factors of H and therefore gets a non-zero answer. Section 5 proved that $\sigma_{16} \cdot T_{16}[A]$ is ANTISYMMETRIC. An antisymmetric

bilinear form vanishes on the diagonal, so

$$\text{Psi16}^T \cdot \text{sigma16} \cdot \text{T16}[\alpha] \cdot \text{Psi16} == 0 \text{ identically,}$$

hence its derivative vanishes too, and therefore

$$\begin{aligned} & \text{Psi16}^T \cdot \text{sigma16} \cdot \text{T16}[\alpha] \cdot D[\text{Psi16}, x_{\alpha}] \\ & == -D[\text{Psi16}, x_{\alpha}]^T \cdot \text{sigma16} \cdot \text{T16}[\alpha] \cdot \text{Psi16} \end{aligned}$$

holds EXACTLY, with no boundary term at all. The kinetic term is its own negative transpose: that is what makes the variational problem well posed. We verify both statements below.

```
ClearAll[La];
La[] := ((1/H) Transpose[Ψ16] . σ16 .
  Sum[T16[α1 - 1] . D[Ψ16, X[[α1]]], {α1, 1, Length[X]}]
  + 2 (M/H) Transpose[Ψ16] . σ16 . Ψ16
) // Simplify[#, constraintVars] &;
cfTimed["evaluate the Lagrangian La[]", La[]] // Short[#, 10] &

(* --- the two exact identities behind the
Lagrangian ----- *)
cfAssert["Psi16 . sigma16 . T16[alpha] . Psi16 == 0 identically (antisymmetric form)",
  Table[Expand[Transpose[Ψ16] . σ16 . T16[α1 - 1] . Ψ16] === 0,
    {α1, 1, 8}]]];
cfAssert["the kinetic term equals minus its transpose, exactly",
  Expand[
    Transpose[Ψ16] . σ16 .
    Sum[T16[α1 - 1] . D[Ψ16, X[[α1]]], {α1, 1, 8}]
    + Sum[D[Transpose[Ψ16], X[[α1]]] . σ16 . T16[α1 - 1],
      {α1, 1, 8}] . Ψ16] === 0];

(* --- so the Lagrangian and its "conjugate"
form differ only in the mass coefficient ----- *)
(* The original's scratch cell compares La
against a conjugate form written with 2 H M *)
(* instead of 2 M/H and without the 1/H on the kinetic term,
so its difference does not *)
(* vanish. Written with matching coefficients
the difference is exactly zero. *)
cfAssert["La[] == its conjugate form when the coefficients match",
  Simplify[La[] - (
    - (1/H) Sum[D[Transpose[Ψ16], X[[α1]]] . σ16 . T16[α1 - 1],
      {α1, 1, 8}] . Ψ16
    + 2 (M/H) Transpose[Ψ16] . σ16 . Ψ16),
  constraintVars] === 0];
```

12. The Euler-Lagrange equations, the coupling pattern, and the (z,t) chart

The Euler-Lagrange operator of the original notebook, transcribed exactly. For each component index k it forms

$$\begin{aligned} & (1/\text{detsqrt}) \left(D[L, f16[k][x0, x4]] \right. \\ & \quad - D[D[L, \text{Derivative}[1, 0][f16[k]][x0, x4]], x0] \\ & \quad \left. - D[D[L, \text{Derivative}[0, 1][f16[k]][x0, x4]], x4] \right) \end{aligned}$$

simplified under `constraintVars`, and then applies `subsDefects`. For the flat Lagrangian `La` the volume factor `detsqrt` is 1.

```
ClearAll[eL];
eL[Lagrangian_Symbol, detsqrt_] := Module[{L, t},
  L = Lagrangian[];
  t = Table[
    FullSimplify[
      (1 / detsqrt) (
        D[L, f16[k][x0, x4]]
        - D[D[L, Derivative[1, 0][f16[k]][x0, x4]], x0]
        - D[D[L, Derivative[0, 1][f16[k]][x0, x4]], x4]),
      constraintVars],
    {k, 0, 15}];
  Return[t /. subsDefects];
];

(* --- the sixteen field
   equations ----- *)
ClearAll[eLa];
eLa = cfTimed["Euler-Lagrange equations eLa = eL[La, 1]", eL[La, 1]];
cfAssert["there are sixteen field equations", Length[eLa] === 16];
Column[eLa]
```

```

(* --- which components are coupled to
    which ----- *)
(* rawSets collects, for each equation,
the set of component indices that appear in it. *)
(* MergeSetsStep merges any two sets that intersect;
iterating it to a fixed point gives the *)
(* partition of the sixteen components into
independent blocks. *)
ClearAll[rawSets, MergeSetsStep, showCoupledEquations];
rawSets[l_, f_, o_: 0] := Table[
  Cases[l[[j]], h_Symbol[n_Integer] /; StringEndsQ[SymbolName[h], f] &=> n + o,
  Infinity, Heads -> True],
  {j, 1, Length[l]}];
MergeSetsStep[currentSets_List] := Module[{i, j, merged = currentSets},
  Catch[
    For[i = 1, i <= Length[merged], i++,
      For[j = i + 1, j <= Length[merged], j++,
        If[Intersection[merged[[i]], merged[[j]]] != {},
          merged = Delete[merged, {{i}, {j}}];
          AppendTo[merged, Sort[Union[currentSets[[i]], currentSets[[j]]]]];
          Throw[Sort[merged]]];
        Sort[merged]]];
  showCoupledEquations[items_List] := FixedPoint[MergeSetsStep, items];

ClearAll[eLaRawSets, eLaCouplings];
eLaRawSets = rawSets[eLa, "f16", 0];
eLaCouplings = cfTimed["partition the 16 components into coupled blocks",
  showCoupledEquations[eLaRawSets]];
cfAssert["the coupled blocks partition all sixteen components",
  Sort[Flatten[eLaCouplings]] === Range[0, 15]];
cfAssert["there are four blocks of four",
  Length /@ eLaCouplings === {4, 4, 4, 4}];
eLaCouplings

(* --- save the equations,
exactly as the original does ----- *)
ClearAll[$cfWriteMX];
$cfWriteMX = True;
If[$cfWriteMX,
  DumpSave[FileNameJoin[{cfDir, cfHeader <> "eLa.mx"}], eLa];
  Print["wrote ", FileNameJoin[{cfDir, cfHeader <> "eLa.mx"}]]];

Now change the chart. Substituting f16[k] -> Z[k][6 H x0, H x4] and then x0 -> z/(6H),
x4 -> t/H turns the sixteen equations into equations for the Z[k] in the coordinates (z,t).
The overall factor 1/(2H) is the original's normalization.

```

```

ClearAll[eLazt];
eLazt = cfTimed["change of chart: eLazt = (1/(2H)) eLa in the (z,t) coordinates",
  ((1/(2H)) eLa /. sfψ16Aa /. sx0x4) // FullSimplify[#, constraintVars] &];
If[$cfWriteMX,
  DumpSave[FileNameJoin[{cfDir, cfHeader <> "eLazt.mx"}], eLazt];
  Print["wrote ", FileNameJoin[{cfDir, cfHeader <> "eLazt.mx"}]]];
Column[eLazt]

```

13. Decoupling into four blocks of four, and the closed-form solutions

The sixteen equations couple the components in four groups of four. Relabelling the components so that each group becomes a consecutive block of four turns the system into four independent 4x4 first-order systems. The relabelling map is

sZtOyZ: Z[eLaCouplings flattened] -> yZ[0], yZ[1], ..., yZ[15],

i.e. the j-th component in the flattened coupling list becomes yZ[j-1].

```

ClearAll[varZ, varZzt, DzvarZ, DtvarZ];
varZ = Table[Z[k], {k, 0, 15}];
varZzt = Through[varZ[z, t]];
DzvarZ = D[varZzt, z];
DtvarZ = D[varZzt, t];
{Short[varZzt, 3], Short[DtvarZ, 3]}

(* --- solve the system for the t-
   derivatives ----- *)
ClearAll[DtvarZsubs, DtvarZeqs, DtvarZrelations];
DtvarZsubs = cfTimed["solve the 16 equations for the t-derivatives",
  Solve[And @@ Thread[0 == eLazt], DtvarZ][[1]] // FullSimplify[#, constraintVars] &];
DtvarZeqs = DtvarZsubs /. {Rule -> Equal};
DtvarZrelations = DtvarZsubs /. {Rule -> Subtract};
cfAssert["all sixteen t-derivatives are determined", Length[DtvarZsubs] === 16];
Column[DtvarZsubs]

```

```

(* --- the relabelling,
and its inverse ----- *)
ClearAll[yZdef, Zdef, sZtOyZ, syZtoZ, zeroZyZforCaExpression, zeroZyZEQS];
yZdef = Table[yZ[k], {k, 0, 15}];
Zdef = Table[Z[k], {k, 0, 15}];
sZtOyZ = Table[Z[Flatten[eLaCouplings][[j]]] → yZ[j - 1], {j, 1, 16}];
syZtoZ = Solve[And @@ (sZtOyZ /. {Rule → Equal}), yZdef][[-1]];
zeroZyZforCaExpression = -(sZtOyZ /. {Rule → Subtract});
zeroZyZEQS = sZtOyZ /. {Rule → Equal};
cfAssert["the relabelling is a bijection of the sixteen components",
Sort[sZtOyZ[[All, 1]]] === Sort[Zdef] && Sort[sZtOyZ[[All, 2]]] === Sort[yZdef]];
Column[sZtOyZ]

(* --- the equations in the new labels,
and the four 4x4 blocks ----- *)
ClearAll[eLyZ, DtyZsubsA, DtvaryZEQS, zeroDtyZeqs, coupledZeqs];
eLyZ = cfTimed["relabel the equations", (eLazt /. sZtOyZ) // FullSimplify];
DtyZsubsA = cfTimed["solve the relabelled system for the t-derivatives",
Solve[And @@ Thread[0 == eLyZ], D[Through[yZdef[z, t]], t]][[1]] //
FullSimplify[#, constraintVars] &];
DtvaryZEQS = DtyZsubsA /. {Rule → Equal};
zeroDtyZeqs = DtyZsubsA /. {Rule → Subtract};
coupledZeqs = Partition[DtvaryZEQS, 4];
cfAssert["the system splits into four blocks of four equations",
Dimensions[coupledZeqs] === {4, 4}];
Column[coupledZeqs]

(* --- each block really is
closed: block j mentions only yZ[4(j-1)] .. yZ[4(j-1)+3] ----- *)
cfAssert["each 4x4 block is closed in its own four components",
Table[SubsetQ[Range[4 (blk - 1), 4 (blk - 1) + 3],
Union@Cases[coupledZeqs[[blk]], yZ[n_Integer] → n, Infinity, Heads → True]],
{blk, 1, 4}]];
Table[Union@Cases[coupledZeqs[[blk]], yZ[n_Integer] → n, Infinity, Heads → True],
{blk, 1, 4}]

```

The original notebook now tries DSolve on each block. We run the same four calls, but wrapped in TimeConstrained so that the notebook cannot hang on them, and we report honestly whether each one returned a closed form or came back unevaluated. The original's experience was that Mathematica does not solve these blocks, which is why the author turned to Maple; the cells below record what actually happens in this kernel rather than assuming either outcome.

```

ClearAll[yZvar, cfDSolveTry, dsolveResults];
yZvar = Through[yZdef[z, t]];
cfDSolveTry[blk_Integer, seconds_ : 60] := Module[{r},
  r = TimeConstrained[
    Quiet@Check[DSolve[coupledYeqs[[blk]], yZvar[[4 (blk - 1) + 1 ;; 4 blk]], {z, t}],
    $cfDSolveFailed],
    seconds, $cfDSolveTimedOut];
Print["  block ", blk, ":", " ",
  Which[r === $cfDSolveTimedOut, "TIMED OUT",
    r === $cfDSolveFailed, "DSolve reported an error",
    MatchQ[r, _DSolve], "returned unevaluated",
    r === {}, "no solution found",
    True, "closed form with " <> ToString[Length[r]] <> " branch(es)"]];
r];
dsolveResults =
  cfTimed["DSolve on the four blocks", Table[cfDSolveTry[blk], {blk, 1, 4}]];
Short[dsolveResults, 8]

```

The closed forms below were obtained with Maple by the author and pasted back into the original notebook as literal strings. They are reproduced here verbatim. The parser that turns them into Wolfram expressions is the author's own ConvertMapleToMathematicaV2, whose load banner warns that it should not be trusted for a correct result. That warning does not matter here, because the solutions it produces are SUBSTITUTED BACK into the four blocks and verified symbolically a few cells below: the verification, not the parser, is what certifies them.

Each block j involves one constant C_j and four constants $c_{j1}..c_{j4}$, and every component carries the same exponential

$$\exp\left(\left(6\left(\frac{z}{6}+t\right)H^2C_j^2-6\left(-\frac{z}{6}+t\right)M^2\right)/\left(6C_jH^2\right)\right).$$

```

ClearAll[maplestringEQ1, maplestringEQ2, maplestringEQ3, maplestringEQ4];
maplestringEQ1 = "{yZ0(z, t) = exp((6*(z/6 + t)*H^2*C1^2 - 6*(-z/6
+ t)*M^2)/(6*C1*H^2))*(H^2*(c11*c12 - c13*c14)*C1^2 - M^2*(c11*c12
+ c13*c14))/(2*H*C1*M), yZ1(z, t) = -(H^2*(c11*c12 - c13*c14)*C1^2
+ M^2*(c11*c12 + c13*c14))*exp((6*(z/6 + t)*H^2*C1^2 - 6*(-z/6
+ t)*M^2)/(6*C1*H^2)))/(2*H*C1*M), yZ2(z, t) = c13*exp((6*(z/6
+ t)*H^2*C1^2 - 6*(-z/6 + t)*M^2)/(6*C1*H^2))*c14, yZ3(z, t) =
c11*exp((6*(z/6 + t)*H^2*C1^2 - 6*(-z/6 + t)*M^2)/(6*C1*H^2))*c12}";
maplestringEQ2 = "{yZ4(z, t) = -exp((6*(z/6 + t)*H^2*C2^2 - 6*(-z/6
+ t)*M^2)/(6*C2*H^2))*(H^2*(c21*c22 + c23*c24)*C2^2 - M^2*(c21*c22
- c23*c24))/(2*H*C2*M), yZ5(z, t) = -(H^2*(c21*c22 + c23*c24)*C2^2
+ M^2*(c21*c22 - c23*c24))*exp((6*(z/6 + t)*H^2*C2^2 - 6*(-z/6
+ t)*M^2)/(6*C2*H^2)))/(2*H*C2*M), yZ6(z, t) = c23*exp((6*(z/6
+ t)*H^2*C2^2 - 6*(-z/6 + t)*M^2)/(6*C2*H^2))*c24, yZ7(z, t) =
c21*exp((6*(z/6 + t)*H^2*C2^2 - 6*(-z/6 + t)*M^2)/(6*C2*H^2))*c22}";
maplestringEQ3 =
"{yZ10(z, t) = c33*exp((6*H^2*(z/6 + t)*C3^2 - 6*(-z/6 + t)*M^2)/(6*C3*H^2))*c34,
yZ11(z, t) = c31*exp((6*H^2*(z/6 + t)*C3^2 - 6*(-z/6 +
t)*M^2)/(6*C3*H^2))*c32, yZ8(z, t) = -exp((6*H^2*(z/6 + t)*C3^2
- 6*(-z/6 + t)*M^2)/(6*C3*H^2))*(H^2*(c31*c32 + c33*c34)*C3^2 -
M^2*(c31*c32 - c33*c34))/(2*H*C3*M), yZ9(z, t) = -(H^2*(c31*c32
+ c33*c34)*C3^2 + M^2*(c31*c32 - c33*c34))*exp((6*H^2*(z/6
+ t)*C3^2 - 6*(-z/6 + t)*M^2)/(6*C3*H^2)))/(2*H*C3*M)}";
maplestringEQ4 = "{yZ12(z, t) = exp((6*H^2*(z/6 + t)*C4^2 - 6*(-z/6 +
t)*M^2)/(6*C4*H^2))*(H^2*(c41*c42 - c43*c44)*C4^2 - M^2*(c41*c42 +
c43*c44))/(2*H*C4*M), yZ13(z, t) = -(H^2*(c41*c42 - c43*c44)*C4^2
+ M^2*(c41*c42 + c43*c44))*exp((6*H^2*(z/6 + t)*C4^2 - 6*(-z/6 +
t)*M^2)/(6*C4*H^2)))/(2*H*C4*M), yZ14(z, t) = c43*exp((6*H^2*(z/6
+ t)*C4^2 - 6*(-z/6 + t)*M^2)/(6*C4*H^2))*c44, yZ15(z, t) =
c41*exp((6*H^2*(z/6 + t)*C4^2 - 6*(-z/6 + t)*M^2)/(6*C4*H^2))*c42}";
StringLength /@ {maplestringEQ1, maplestringEQ2, maplestringEQ3, maplestringEQ4}

(* --- parse
   them ----- *)
ClearAll[solvedEQ, YZvar, sYZvar];
If[! StringQ[cfMaplePath],
Print[
" CANNOT PARSE THE MAPLE SOLUTIONS: ConvertMapleToMathematicaV2.wl was not found.];
Print[" Nothing is substituted for
it. Place the file next to this notebook and re-run."]];
solvedEQ = Table[ConvertMapleToMathematicaV2[
{maplestringEQ1, maplestringEQ2, maplestringEQ3, maplestringEQ4}[[j]], {j, 1, 4}];
YZvar = Table[ToExpression["YZ" <> ToString[k] <> "[z,t]"], {k, 0, 15}];
sYZvar = Solve[And @@ Flatten[solvedEQ], YZvar][[1]];
cfAssert["all sixteen components have a closed form", Length[sYZvar] == 16];
Column[sYZvar]

```

```

(* --- turn them into pure functions for
yZ ----- *)
ClearAll[ssyZ];
ssyZ = Table[
  yZ[k] → ToExpression[
    "(" <> ToString[FullForm[sYZvar[[k + 1, 2]] /. {z → #1, t → #2}]] <> "&)"],
  {k, 0, 15}];
cfAssert["there are sixteen pure-function rules", Length[ssyZ] === 16];
Short[ssyZ, 6]

(* --- VERIFY the Maple solutions against the
four blocks ----- *)
(* This is the cell that certifies Section
13. Every one of the sixteen equations must *)
(* reduce to True after the closed forms are
substituted. *)
ClearAll[cfMapleCheck];
cfMapleCheck = cfTimed["substitute the closed forms back into the four blocks",
  Table[FullSimplify[coupledyZeqs[[blk]] /. ssyZ], {blk, 1, 4}]];
cfAssert["THE MAPLE CLOSED FORMS SOLVE ALL SIXTEEN EQUATIONS",
  Union[Flatten[cfMapleCheck]] === {True}];
cfMapleCheck

(* --- back to the original labels,
and to Psi16 ----- *)
ClearAll[sZ,  $\Psi$ 16a];
sZ = Sort[sZt0yZ /. ssyZ];
 $\Psi$ 16a = cfTimed["Psi16 with the closed forms substituted",
  ( $\Psi$ 16 /. sf $\Psi$ 16Aa /. sx0x4 /. sZ /. sZt) // FullSimplify];
Column[ $\Psi$ 16a]

```

One more structural result of the original notebook. The relabelling $Z \rightarrow yZ$ is a linear map; extracting its matrix with CoefficientArrays shows that the map is an ORTHOGONAL 16x16 matrix, so it preserves the Euclidean form, but it does NOT preserve sigma16, so it is not a Spin(8,8) transformation; and it is not block diagonal, so it mixes the type-1 and type-2 spinors.

```

ClearAll[cayZ, caZ, caZ2, almightyS];
cayZ = CoefficientArrays[zeroZyZforCaExpression, yZdef];
caZ = CoefficientArrays[zeroZyZforCaExpression, Zdef];
caZ2 = Normal[caZ[[2]]];
almightyS = -Transpose[caZ2];
cfAssert["the yZ coefficient matrix is the identity", Normal[cayZ[[2]]] === ID16];
cfAssert["ORTHOGONAL: Transpose[S] . S == ID16",
  Transpose[almightyS] . almightyS === ID16];
cfAssert["ORTHOGONAL: S . Transpose[S] == ID16",
  almightyS . Transpose[almightyS] === ID16];
cfAssert["but NOT a Spin(8,8) transformation: it does not preserve sigma16",
  Transpose[almightyS] .  $\sigma_{16}$  . almightyS !=  $\sigma_{16}$ ];
cfAssert["and NOT a direct sum: it mixes type-1 with type-2",
  caZ2 != ArrayFlatten[{{caZ2[[1 ;; 8, 1 ;; 8]], 0}, {0, caZ2[[9 ;; 16, 9 ;; 16]]}}]];
MatrixForm[caZ2]

```

14. Bilinears, the two solution branches, and M6 = 3 generations of Einstein-Rosen 2-planes

Five bilinear invariants are formed from the solution. A word first on WHICH split of the sixteen components is used, because it is not the one the names suggest.

psisol below is the solution written in the RELABELLED yZ basis of Section 12, and psi1sol and psi2sol are its first and second blocks of eight in THAT basis. They are NOT the type-1 and type-2 split-octonion spinors. The relabelling regroups the sixteen components by which four of them couple to each other in the field equations, and Section 13 has already PROVED that the regrouping mixes the two types: the assertion "and NOT a direct sum: it mixes type-1 with type-2" is exactly that check. The relabelling table printed in Section 12 shows it explicitly,

$$yZ[0..7] == \{Z_0, Z_5, Z_8, Z_{13}, Z_1, Z_4, Z_9, Z_{12}\}$$

which is four type-1 components (Z0..Z7) and four type-2 components (Z8..Z15). The cell below asserts both of these facts rather than leaving them to the reader. The genuine type-1 and type-2 spinors appear two cells later as psi1 = Psi16ab[[1;;8]] and psi2 = Psi16ab[[9;;16]]; the names are similar and the objects are not, so the two are kept distinct here on purpose.

The five bilinears are then

```

psisol . sigma16 . psisol  (the full 16-component norm)
psi1sol . sigma  . psi1sol  (the norm of the first relabelled block)
psi2sol . sigma  . psi2sol  (the norm of the second relabelled block)
psi2sol . psi1sol          (the unpaired product)

```

$\psi_2 \text{sol} \cdot \sigma_{16} \cdot \psi_1 \text{sol}$ (the paired product)

A correction to a claim this notebook used to make about the first of them. Section 11 proved that $\psi \cdot \sigma_{16} \cdot T_{16}[\alpha] \cdot \psi$ vanishes for every α -- WITH the T_{16} factor, which is what makes $\sigma_{16} \cdot T_{16}[\alpha]$ antisymmetric and the kinetic term self-adjoint. σ_{16} by itself is symmetric (Section 5), so $\psi \cdot \sigma_{16} \cdot \psi$ is not forced to vanish, and for this solution it does not: it is the difference of the two block norms. Both statements are asserted below.

Requiring the two block norms to be constant along the light cone is what fixes two of the four block constants. We first introduce the advanced and retarded light-cone coordinates in the (z, t) chart, which are the original notebook's z_{adv} and z_{ret} .

```
ClearAll[sztar, sszt, ztadv, ztret];
sztar = Solve[ztadv == 6 t + z && ztret == 6 t - z, {ztadv, ztret}][[1]];
sszt = Solve[ztadv == 6 t + z && ztret == 6 t - z, {z, t}][[1]];
{sztar, sszt}

(* --- the five
   bilinears ----- *)
ClearAll[ψsol, ψ1sol, ψ2sol, psiSigma16psi, psi1Sigmapsi1, psi2Sigmapsi2,
  psi2psi1, psi2Sigmapsi1];
ψsol = Through[yZdef[z, t]] /. ssyZ;
ψ1sol = ψsol[[1 ;; 8]];
ψ2sol = ψsol[[9 ;; 16]];
(* The split at 8 is a split of the yZ COUPLING blocks,
   NOT of the type-1/type-2 direct sum. *)
(* Section 13 proved the relabelling mixes the two types;
   these two assertions say exactly how, *)
(* so that the names psi1sol/psi2sol cannot be misread as the split-
   octonion halves. *)
cfAssert["the yZ split at 8 is NOT the type-1/type-2 split",
  Sort[sZtOyZ[[1 ;; 8, 1]]] != Sort[Table[Z[k], {k, 0, 7}]]];
cfAssert["the first relabelled block holds four type-1 and four type-2 components",
  {Count[sZtOyZ[[1 ;; 8, 1]], Z[k_Integer] /; k ≤ 7],
   Count[sZtOyZ[[1 ;; 8, 1]], Z[k_Integer] /; k ≥ 8]} == {4, 4}];
(* FullSimplify::time is EXPECTED on two of
   these five bilinears: Section 1 set the original *)
(* notebook's 3-
   second FullSimplify budget and these expressions exceed it. FullSimplify then *)
(* returns the best form it reached,
   which is correct but not maximally simplified, so the *)
(* message is informational. It is suppressed here and only here,
   and announced first. *)
Print[" EXPECTED MESSAGE FullSimplify::time
      may fire on the bilinears; the 3-second budget"];
```

```

Print["          set in
      Section 1 is the original notebook's, and a budget that is"];
Print["          reached
      still returns a correct, merely less-simplified, result."];
psiSigma16psi = cfTimed["bilinear psi . sigma16 . psi",
  Quiet[ $\psi_{sol} . \sigma_{16} . \psi_{sol}$  // FullSimplify[#, constraintVars] &,
    {FullSimplify::time, General::stop}]]];
psi1Sigmapsi1 = cfTimed["bilinear psi1 . sigma . psi1",
  Quiet[ $\psi_{1sol} . \sigma . \psi_{1sol}$  // FullSimplify[#, constraintVars] &,
    {FullSimplify::time, General::stop}]]];
psi2Sigmapsi2 = cfTimed["bilinear psi2 . sigma . psi2",
  Quiet[ $\psi_{2sol} . \sigma . \psi_{2sol}$  // FullSimplify[#, constraintVars] &,
    {FullSimplify::time, General::stop}]]];
psi2psi1 = cfTimed["bilinear psi2 . psi1",
  Quiet[ $\psi_{2sol} . \psi_{1sol}$  // FullSimplify[#, constraintVars] &,
    {FullSimplify::time, General::stop}]]];
psi2Sigmapsi1 = cfTimed["bilinear psi2 . sigma . psi1",
  Quiet[ $\psi_{2sol} . \sigma . \psi_{1sol}$  // FullSimplify[#, constraintVars] &,
    {FullSimplify::time, General::stop}]]];
(* sigma16 == ArrayFlatten[{{-sigma,0},{0,sigma}}],
so the 16-component norm is the DIFFERENCE *)
(* of the two 8-component norms. Note that sigma16 itself is SYMMETRIC,
so this bilinear does *)
(* not vanish; it is sigma16 . T16[A] that is antisymmetric,
which is what made the kinetic *)
(* term of Section 11 self-
adjoint. *)
cfAssert["psi . sigma16 . psi == psi2.sigma.psi2 - psi1.sigma.psi1",
  Simplify[psiSigma16psi - (psi2Sigmapsi2 - psi1Sigmapsi1), constraintVars] === 0];
(* This notebook used to describe psi . sigma16 . psi
as "identically zero, as Section 11 *)
(* showed". That was wrong on both
counts and is corrected here. Section 11's identity is *)
(* psi . sigma16 . T16[alpha] . psi == 0,
with the T16 factor. Without it the form is *)
(* symmetric, so nothing forces the bilinear
to vanish -- and it does not. *)
cfAssert["sigma16 alone is SYMMETRIC, so psi . sigma16 . psi is not forced to vanish",
  Transpose[ $\sigma_{16}$ ] ===  $\sigma_{16}$ ];
cfAssert["and for this solution it does NOT vanish: it is not the zero expression",
  Simplify[psiSigma16psi, constraintVars] != 0];
Column[{psiSigma16psi, psi1Sigmapsi1, psi2Sigmapsi2, psi2psi1, psi2Sigmapsi1}]

```

```

(* --- demanding that the two norms be light-
cone constants fixes C2 and C4 ----- *)
ClearAll[cfNormDerivs, cfConstantSolutions];
cfNormDerivs = Union@Flatten@Table[
  {Simplify[D[b /. sszt, ztadv], constraintVars],
   Simplify[D[b /. sszt, ztret], constraintVars]},
  {b, {psi1Sigmapsi1, psi2Sigmapsi2}}];
(* Solve::ifun and Solve::svars are EXPECTED
here. The conditions are transcendental in the *)
(* block constants,
so Solve inverts an exponential (ifun) and does not determine every one of *)
(* the four constants (svars) -- a one-parameter family survives,
which is the point. The *)
(* branch that Solve returns is verified
independently in the next cell. *)
Print[
  " EXPECTED MESSAGES Solve::ifun and Solve::svars: the light-cone conditions are"];
Print["
transcendental
and leave a family of solutions, not a unique one."];
cfConstantSolutions = Quiet[Solve[And @@ Thread[0 == cfNormDerivs], {C1, C2, C3, C4}],
  {Solve::ifun, Solve::svars}];
cfAssert["the light-cone-constant condition has solutions",
  cfConstantSolutions != {}];
{cfNormDerivs, cfConstantSolutions}

(* --- the original's branch: C2 →
-C1 and C4 → -C3 ----- *)
ClearAll[cfBranch, cfBilinearsOnBranch];
cfBranch = {C2 → -C1, C4 → -C3};
cfBilinearsOnBranch = Simplify[
  {psiSigma16psi, psi1Sigmapsi1, psi2Sigmapsi2, psi2Sigmapsi1} /. sszt /. cfBranch,
  constraintVars];
cfAssert[
  "on the branch C2 = -C1, C4 = -C3 the two norms are independent of ztadv and ztret",
  Flatten@Table[Simplify[{D[b, ztadv], D[b, ztret]}, constraintVars] == {0, 0},
  {b, cfBilinearsOnBranch[[2 ;; 3]]}];
Column[cfBilinearsOnBranch]

```

```

(* --- the two mass branches of the original
notebook ----- *)
(* Psi16ab keeps a free dimensionless mass ratio Mab;
Psi16aa is the special case Mab = 1. *)
ClearAll[ψ16ab, ψ16aa, ψ16x0ONLY];
ψ16ab = cfTimed["Psi16ab (M -> H Mab Sqrt[C1 C3])",
(ψ16a /. sfψ16Aa /. sx0x4 /. sZ /. sz
 /. {C2 -> -C1, C4 -> -C3, M -> H Mab Sqrt[C1 C3]}) // FullSimplify];
ψ16aa = cfTimed["Psi16aa (M -> H Sqrt[C1 C3])",
(ψ16a /. sfψ16Aa /. sx0x4 /. sZ /. sz
 /. {C2 -> -C1, C4 -> -C3, M -> H Sqrt[C1 C3]}) // FullSimplify];
ψ16x0ONLY = cfTimed["Psi16x0ONLY (C3 -> C1)",
(ψ16aa /. {C3 -> C1}) // FullSimplify[#, C1 > 0] &];
Column[{Short[ψ16ab, 6], Short[ψ16aa, 6], Short[ψ16x0ONLY, 6]}]

(* --- advanced and retarded coordinates in
the (x0,x4) chart ----- *)
ClearAll[sξ, ssx0x4, ξadv, ξret];
sξ = Solve[ξadv == x0 - x4 && ξret == x0 + x4, {ξadv, ξret}][[1]];
ssx0x4 = Solve[ξadv == x0 - x4 && ξret == x0 + x4, {x0, x4}][[1]];
{sξ, ssx0x4}

(* --- the two split-
octonion spinors of the solution ----- *)
ClearAll[ψ1, ψ2, type1DIVBYtype2, realPartOfType1, realPartOfType2];
ψ1 = FullSimplify[ψ16ab[[1 ;; 8]]];
ψ2 = FullSimplify[ψ16ab[[9 ;; 16]]];
type1DIVBYtype2 = FullSimplify[ψ16ab[[1 ;; 8]] / ψ16ab[[9 ;; 16]]];
realPartOfType1 = FullSimplify[unit . ψ1];
realPartOfType2 = FullSimplify[unit . ψ2];
Column[{Short[ψ1, 4], Short[ψ2, 4], Short[type1DIVBYtype2, 4],
realPartOfType1, realPartOfType2}]

```

Triality now turns each spinor into a VECTOR of the split octonion algebra. Using the constant bridge of Section 9,

```

vectorFormOfType2 = triVecToSpin . psi2
vectorFormOfType1 = eta4488 . (Transpose[psi1] . sigma . triSpinToVec)

```

Which of the two mutually inverse bridge matrices goes on which side is not a matter of taste. `triVecToSpin[[A,a]]` carries a VECTOR row index and a SPINOR column index, so LEFT-multiplying a column spinor by it produces a vector, while RIGHT-multiplying a row spinor by `triSpinToVec` does the same on the other side. Exchanging the two would still type-check, would still pass every assertion attached to these objects (any non-zero covector annihilates some 7-plane), and would silently produce different 7-planes and a different M6. These are the objects the original notebook writes as F with indices a,A and A,a respectively, in exactly this arrangement.

Each of these two vectors is normal to a 7-plane through the origin of the split octonion algebra, namely the set of position vectors Z_{oct} with $\langle \text{vector}, Z_{oct} \rangle = 0$. Two 7-planes that are not parallel meet in a 6-plane. That 6-plane, called M6 in the original notebook, is the object of the title: the three generations of Einstein-Rosen 2-plane bridges live in it.

A naming note. The original notebook writes the 8-component position vector as Z , which collides with the head $Z[k]$ used for the wave-function components earlier in the same session. We call it Z_{oct} here. The mathematics is unchanged; only the symbol is.

```
ClearAll[vectorFormOfType1, vectorFormOfType2];
(* triVecToSpin[[A,a]]: vector row,
spinor column. It maps a column SPINOR to a VECTOR from *)
(* the left; triSpinToVec does the reverse
and acts on a ROW spinor from the right. *)
vectorFormOfType2 = FullSimplify[triVecToSpin .  $\psi_2$ ];
vectorFormOfType1 = FullSimplify[ $\eta_{4488}$  . (Transpose[ $\psi_1$ ] .  $\sigma$  . triSpinToVec)];
cfAssert["both triality vectors have eight components",
{Length[vectorFormOfType1], Length[vectorFormOfType2]} == {8, 8}];
(* the index placement is not arbitrary: each bridge must be used on its own side *)
cfAssert["vectorFormOfType2 ==  $\psi_2$  contracted on the SPINOR index of triVecToSpin",
Simplify[vectorFormOfType2
- Table[Sum[triVecToSpin[[A1, a1]]  $\psi_2$ [[a1]], {a1, 8}], {A1, 8}]] ==
ConstantArray[0, 8]];
cfAssert["triVecToSpin and triSpinToVec really are mutually inverse",
triVecToSpin . triSpinToVec == ID8];
Column[{Short[vectorFormOfType1, 5], Short[vectorFormOfType2, 5]}]

(* --- the two 7-
planes ----- *)
ClearAll[Zoct, zoct, sz8, sz7, sevenPlane1, sevenPlane2];
Zoct = Array[zoct, 8];
sz8 = FullSimplify[Solve[0 == vectorFormOfType2 . Zoct, zoct[8]]][[-1]];
sz7 = Simplify[Solve[0 == vectorFormOfType1 . Zoct, zoct[7]]][[-1]];
sevenPlane2 = Zoct /. sz8;
sevenPlane1 = Zoct /. sz7;
cfAssert["sevenPlane2 is annihilated by vectorFormOfType2",
Simplify[vectorFormOfType2 . sevenPlane2] == 0];
cfAssert["sevenPlane1 is annihilated by vectorFormOfType1",
Simplify[vectorFormOfType1 . sevenPlane1] == 0];
Column[{Short[sevenPlane1, 5], Short[sevenPlane2, 5]}]
```

```

(* --- and their intersection,
the 6-plane M6 ----- *)
ClearAll[sixPlane78, M6];
(* Solve::svars is EXPECTED here and is in
fact the result we want: intersecting two 7-planes *)
(* in an 8-
space determines only TWO of the eight components and leaves SIX free. That is the *)
(* 6-plane M6. The free-
parameter count is asserted immediately below. *)
Print[" EXPECTED MESSAGE Solve::svars:
      two 7-planes fix only two of the eight components,"];
Print["      leaving the six free parameters of M6."];
sixPlane78 = cfTimed["intersect the two 7-planes",
Simplify[
  Quiet[Solve[And @@ Thread[sevenPlane1 == sevenPlane2], Zoct], {Solve::svars}]]];
M6 = cfTimed["M6 in advanced / retarded coordinates",
Simplify[Zoct /. sixPlane78 /. ssx0x4 /. sξ]];
cfAssert["M6 is a non-empty solution set", sixPlane78 != {}];
cfAssert["M6 has six free parameters",
Length[Complement[Array[zoct, 8], First[sixPlane78][[All, 1]]] == 6];
{Short[sixPlane78, 8], Short[M6, 8]}

(* --- the free directions of M6,
i.e. the six parameters that survive ----- *)
Grid[Prepend[
  Table[{zoct[j],
    If[MemberQ[First[sixPlane78][[All, 1]], zoct[j]], "determined", "free parameter"]},
    {j, 1, 8}], {"component", "status"}], Frame → All, Alignment → Left]

```

Finally, the frame-resolution check of the original notebook.

The original defines two helper functions, `resolution` and `resolution2`, in two cells that are laid out as several side-by-side columns with pasted images used as superscripts inside `Print` strings. That layout cannot be transcribed into linear code unambiguously, so we do NOT guess at the original text. What those cells check is unambiguous, and it is re-implemented here directly: for a spinor v of non-zero norm, the frame built from v by triality must agree with the closed-form bridge of Section 9, and the two bridges must be mutually inverse.

```

ClearAll[cfResolution];
cfResolution[v_List] := Module[{u, EAa, EaA, FaA},
  If[Length[v] != 8,
    Print[" ERROR cfResolution expects a length-8 spinor; got ", Length[v]];
    Return[$Failed]];
  If[Simplify[Transpose[v] .  $\sigma$  . v] === 0,
    Print[" ERROR cfResolution needs a spinor of non-zero sigma-norm"];
    Return[$Failed]];
  u = FullSimplify[v / Sqrt[Abs[Transpose[v] .  $\sigma$  . v]]];
  EAa = Table[u .  $\sigma$  .  $\tau$ bar[A1], {A1, 0, 7}];
  EaA = FullSimplify[Inverse[EAa]];
  FaA = FullSimplify[Transpose[Table[ $\eta_{4488}$ [A1 + 1, A1 + 1] ( $\tau$ [A1] . u), {A1, 0, 7}]]];
  <|"EAa" → EAa, "EaA" → EaA, "FaA" → FaA,
    "closedFormMatchesInverse" → (FullSimplify[FaA - EaA] === Zero8),
    "bridgesAreInverse" → (FullSimplify[EAa . EaA] === ID8) |>;
ClearAll[cfResolutionUnit];
cfResolutionUnit = cfResolution[unit];
cfAssert["cfResolution[unit]: the closed-form bridge equals the matrix inverse",
  cfResolutionUnit["closedFormMatchesInverse"]];
cfAssert["cfResolution[unit]: the two bridges are mutually inverse",
  cfResolutionUnit["bridgesAreInverse"]];
cfAssert["cfResolution[unit] reproduces triVecToSpin",
  cfResolutionUnit["EAa"] === triVecToSpin];
Keys[cfResolutionUnit]

```

PART III -- The curved 4+4 spacetime, its frame field, and the canonical spin connection

15. The local flat 4+4 Minkowski coordinate system at every spacetime point

This is the part of the construction that the original notebook records as a result but never derives. We set it out completely.

At every point of the curved 4+4 spacetime there is a local flat 4+4 Minkowski coordinate system. The object that installs it is the FRAME FIELD, also called the vielbein: in eight

dimensions a vielbein is simply an 8-dimensional vierbein, i.e. an 8-dimensional frame field. Writing μ, ν for curved spacetime indices and a, b for flat tangent-space indices, the frame field $e[\mu, a]$ is the bridge

```
[a] curved metric      g[mu,nu]
[b] flat Minkowski metric eta[a,b] (here eta4488, signature 4+4)
[c] frame field       e[mu,a]
[d] the bridge        g[mu,nu] == e[mu,a] eta[a,b] e[nu,b]
```

In Wolfram arrays we store the frame as an 8x8 matrix whose ROW index is the curved index μ and whose COLUMN index is the flat index a , so [d] is the single matrix statement

```
g == frame . eta4488 . Transpose[frame] .
```

The inverse frame (the coframe) is the matrix inverse, stored with the flat index on the row:

```
coframe == Inverse[frame], coframe[[a,nu]] = e_a{}^nu,
coframe . frame == frame . coframe == ID8 .
```

```
(* --- a notice helper,
used whenever we meet a symbol the original never defines ----- *)
ClearAll[cfNote];
cfNote[title_String, body_String] := (
  Print[" NOTE ", title];
  Print["      ", body];
  Style[Column[{Style[title, Bold], body}], Background -> LightYellow]);

(* --- HARD REQUIREMENT: report, do not invent,
every symbol the original leaves undefined -- *)
cfNote["a4 is an undefined scalar function inherited from the original notebook",
"The canonical frame field below
  contains Exp[-a4[H x4]] and Exp[+a4[H x4]]. The original \
notebook never gives a4 a definition;
  it appears only inside the recorded value of the frame \
field. We therefore carry a4 through
  symbolically and we do NOT invent a value for it. Every \
result in Parts III and IV is valid
  for an arbitrary differentiable a4. The same applies to \
la, which occurs only inside the assumption ssX of Section 2."];
```

The canonical frame field of this geometry. It is the diagonal 8x8 matrix recorded in the original notebook as the value of the symbol written there "gtrye with indices alpha and (A)":

```
diag( Tan[6 H x0], q, q, q, 1, p, p, p )
```

$$q = \text{Exp}[-a4[H x4]] / \text{Sin}[6 H x0]^{(1/6)} \quad (\text{the three spacelike 3-space directions})$$

$$p = \text{Exp}[+a4[H x4]] / \text{Sin}[6 H x0]^{(1/6)} \quad (\text{the three superluminal deflating directions})$$

The hidden-space direction $x0$ carries $\text{Tan}[6 H x0]$ and the time direction $x4$ carries 1. Since $\text{eta4488} = \text{diag}(1,1,1,1,-1,-1,-1,-1)$, the line element is

$$\begin{aligned} ds^2 = & \text{Tan}[6 H x0]^2 dx0^2 \\ & + q^2 (dx1^2 + dx2^2 + dx3^2) \\ & - dx4^2 \\ & - p^2 (dx5^2 + dx6^2 + dx7^2). \end{aligned}$$

Note the two exponentials are reciprocal: $q p = 1/\text{Sin}[6 H x0]^{(1/3)}$. The 3-space directions contract exactly as fast as the deflating directions expand. That reciprocity is the geometric content of "pre-gravity, pre-Big-Bang" in this model, and it is what makes the Einstein-Rosen pairing of Section 19 natural.

```
ClearAll[cfQminus, cfQplus, frameCanonical, coframeCanonical, gCanonical, gInvCanonical];
cfQminus = Exp[-a4[H x4]] / Sin[6 H x0]^(1/6);
cfQplus = Exp[+a4[H x4]] / Sin[6 H x0]^(1/6);
frameCanonical = DiagonalMatrix[{Tan[6 H x0],
  cfQminus, cfQminus, cfQminus, 1, cfQplus, cfQplus, cfQplus}];
MatrixForm[frameCanonical]

(* --- assumptions used by every Simplify in
Parts III and IV ----- *)
(* These are the original notebook's assumptions ssX, unchanged. *)
ClearAll[cfGeomAssume, cfSimp, cfSimpArray];
cfGeomAssume = ssX;
cfSimp[expr_] := Simplify[expr, cfGeomAssume];
(* The level specification matters. Map[f, arr, {-1}]
would map f over the ATOMS of arr -- *)
(* the bare symbols and integers buried
inside each entry -- and would leave every compound *)
(* entry untouched,
making this function a silent no-op. ArrayDepth[arr] is the level of *)
(* the entries themselves: 2 for a matrix,
3 for a connection, 4 for a curvature 2-form. *)
cfSimpArray[arr_] := Map[If[# === 0, 0, cfSimp[#]] &, arr, {ArrayDepth[arr]}];
{cfGeomAssume,
(* a one-line self-test of the level specification *)
cfSimpArray[{{Sin[x0]^2 + Cos[x0]^2 - 1, 0}}] === {{0, 0}}}
```

A note on how identities are certified in Parts III and IV.

Section 1 set the original notebook's session policy, Simplify with a 1-second budget per sub-problem. That budget is right for the algebra of Parts I and II, where every entry is an

exact integer or a short rational, but it is not always enough for the curved-space expressions here, which mix $\tan$, $\sin^{1/6}$, $\exp$ of an undetermined function a_4 , and $\cosh/\sinh$ of an undetermined rapidity. Rather than raise the global budget -- which would change the original's session policy and make the whole notebook slow -- we certify an identity in three escalating stages:

1. exact structural equality (the entry is literally 0);
2. Simplify under the geometric assumptions, with a 5-second budget;
3. a high-precision numerical certificate: substitute concrete probe functions for the undetermined a_4 and rapidity, evaluate at three rational sample points inside the assumed domain to 30 significant digits with up to 200 digits of extra working precision, and accept the result as zero only if its magnitude is below 10^{-22} .

Stage 3 uses probe functions ONLY as test inputs. They are never definitions: a_4 and the rapidity remain undetermined everywhere else in this notebook, and no stated result depends on the particular probes.

Stage 3 is reached with two opposite intentions, and this notebook counts them separately because a single number lumping them together was actively misleading.

`$cfNumericCertificates` counts the times stage 3 returned TRUE, i.e. an identity that Simplify could not close and that is therefore accepted on numerical evidence alone. This is the number that measures how much of the notebook is NOT symbolic, and the smaller it is the better.

`$cfNonVanishingWitnesses` counts the times stage 3 returned FALSE inside an assertion of the form `! TrueQ[cfZeroArrayQ[...]]`, which is how this notebook states that something is NOT identically zero -- that the Bridge 3 torsion really is present, that the triality Dirac matrices really do differ from the vector ones, that Λ really is a boost and not a rotation. A single sample point at which an expression evaluates to a non-zero number is a COMPLETE proof that it is not identically zero, so these are not weakened results; they are the strongest form the claim can take.

Both are printed by the final summary. At the time of writing the run reports `$cfNumericCertificates == 0` and `$cfNonVanishingWitnesses == 6`: every identity in this notebook is closed symbolically, and stage 3 is used only to witness the six non-vanishing claims.

One detail about stage 3. Evaluating an expression that is IDENTICALLY ZERO in arbitrary precision always ends by raising `N::meprec`, because no amount of extra working precision can

produce the requested significant digits of zero. That message is therefore the expected signal of success, not a failure, and it is the only message cfZeroQ suppresses. The decision is made by Chop, not by the message: the numbers are computed to 30 significant digits with up to 200 digits of extra working precision, and anything larger than 10^{-22} in magnitude is reported as non-zero.

```
ClearAll[cfProbeRules, cfProbePoints, cfZeroQ, cfZeroArrayQ,
  $cfNumericCertificates, $cfNonVanishingWitnesses];
$cfNumericCertificates = 0;
(* stage 3 said "yes, zero" -- an identity taken on numerics *)
$cfNonVanishingWitnesses = 0;
(* stage 3 said "no, non-zero" -- a witness of non-vanishing *)
(* probe functions: TEST INPUTS ONLY, never definitions *)
cfProbeRules = {
  a4 → Function[u, 3/7 + u/5 + Sin[u]/11],
  cfBoostRapidity → Function[{u, v}, u/3 - 2v/5 + Sin[u + v]/7];
(* sample points chosen inside the assumed domain: H>0, x>0 and 0 < 6 H x0 < Pi/2 *)
cfProbePoints = {
  {H → 1/10, x0 → 1/4, x1 → 1/3,
    x2 → 2/5, x3 → 3/7, x4 → 1/2, x5 → 5/9, x6 → 4/7,
    x7 → 6/11, λOct → 7/13},
  {H → 1/7, x0 → 1/5, x1 → 2/7,
    x2 → 1/6, x3 → 5/8, x4 → 2/3, x5 → 3/8, x6 → 7/9,
    x7 → 1/4, λOct → -3/5},
  {H → 2/9, x0 → 1/8, x1 → 3/5,
    x2 → 4/9, x3 → 1/2, x4 → 3/4, x5 → 2/5, x6 → 1/3,
    x7 → 5/7, λOct → 11/4}};
cfProbePoints[All, 1]
```

```

(* --- the three-
stage zero test ----- *)
(* cfNumericallyZeroQ is stage 3. N::meprec
is the expected signal that an expression is *)
(* identically zero, so it is suppressed here and ONLY here;
Chop makes the decision. *)
ClearAll[cfNumericallyZeroQ];
cfNumericallyZeroQ[expr_] := Block[{$MaxExtraPrecision = 200},
AllTrue[cfProbePoints,
Function[pt,
TrueQ[Max[Abs[Quiet[Chop[N[Flatten[{expr}] /. cfProbeRules /. pt, 30], 10^-22],
{N::meprec, General::stop}]]] == 0]]]];
cfZeroQ[e_] := Module[{s, r},
If[e === 0, Return[True]];
s = Simplify[e, cfGeomAssume, TimeConstraint → 5];
If[s === 0, Return[True]];
r = cfNumericallyZeroQ[s];
(* Count the two outcomes
apart. Incrementing one counter BEFORE the call, as this cell *)
(* used to,
made every non-vanishing assertion look like an identity accepted on numerics. *)
If[r, $cfNumericCertificates++, $cfNonVanishingWitnesses++];
r];
cfZeroArrayQ[arr_] := Module[{flat = Flatten[{arr}], s},
(* stage 1 *)
If[AllTrue[flat, # === 0 &], Return[True]];
(* stage 2: Simplify the whole array at once,
under the geometric assumptions. Skipping *)
(* this and going straight to the
numerical sweep would leave every array identity in *)
(* Parts III and IV certified only at sample points,
which is weaker than the policy this *)
(* notebook
states. *)
s = Simplify[flat, cfGeomAssume, TimeConstraint → 5];
If[AllTrue[s, # === 0 &], Return[True]];
(* stage 3: one fast numerical sweep over whatever stage 2 could not reduce *)
If[cfNumericallyZeroQ[s], $cfNumericCertificates++; Return[True]];
(* still inconclusive: test entry by entry so we learn which entries fail *)
AllTrue[s, cfZeroQ]];
{cfZeroQ[0], cfZeroQ[Tan[6 H x0] Cot[6 H x0] - 1], cfZeroQ[x0 - x4],
cfZeroArrayQ[{{0, Sin[x0]^2 + Cos[x0]^2 - 1}, {Exp[a4[H x4]] Exp[-a4[H x4]] - 1, 0}}]}

(* --- [d] the bridge g =
e . eta . Transpose[e] ----- *)
gCanonical = cfSimp[frameCanonical .  $\eta$ 4488 . Transpose[frameCanonical]];
coframeCanonical = cfSimp[Inverse[frameCanonical]];

```

```

gInvCanonical = cfSimp[Inverse[gCanonical]];
(* Grading these honestly,
because the obvious one is a tautology. *)
(* gCanonical was DEFINED one line above as frame . eta . Transpose[frame],
so asserting *)
(* that it equals frame . eta . Transpose[frame]
restates the definition and cannot fail. *)
(* It is kept because it puts the bridge [d] on screen next to the metric,
but it is *)
(* labelled for what it is. The check with
content is the one after it: that the frame *)
(* the ORIGINAL notebook recorded reproduces
exactly the line element stated in the text *)
(* above. A mistranscribed frame entry
would fail that and pass the tautology. *)
cfAssert["CANONICAL [structural]: frame .
coframe == ID8 (certifies only that the frame is invertible)",
cfZeroArrayQ[frameCanonical . coframeCanonical - ID8]];
cfAssert[
"CANONICAL [definition]: g == frame . eta4488 . Transpose[frame] (bridge [d];
this IS how g was defined, so it cannot fail)",
cfZeroArrayQ[gCanonical - frameCanonical .  $\eta$ 4488 . Transpose[frameCanonical]]];
cfAssert["CANONICAL [has content]: g == diag(Tan[6 H x0]^2,
q^2, q^2, q^2, -1, -p^2, -p^2, -p^2), the stated line element",
cfZeroArrayQ[gCanonical - DiagonalMatrix[{Tan[6 H x0]^2,
cfQminus^2, cfQminus^2, cfQminus^2, -1, -cfQplus^2, -cfQplus^2, -cfQplus^2}]]];
(* Signature. Because the frame is diagonal, g[[i,i]] == eta[[i,i]] frame[[i,i]]^2 exactly, *)
(* which is Sylvester's law of inertia made
explicit: g inherits eta's signature from any *)
(* real invertible frame. That identity
needs no assumption at all. *)
cfAssert["CANONICAL [has content]:
Diagonal[g] == Diagonal[eta4488] * Diagonal[frame]^2 exactly",
cfZeroArrayQ[Diagonal[gCanonical] - Diagonal[ $\eta$ 4488] Diagonal[frameCanonical]^2]];
(* Turning that into "four plus, four minus"
needs two things the notebook cannot derive, *)
(* because a4 is deliberately left undefined: that a4 is REAL-valued,
and that 6 H x0 lies *)
(* in the first quadrant so that Tan[6 H x0] >
0. Both are stated here rather than assumed *)
(* silently. Without them Mathematica cannot
decide the sign of Exp[a4[H x4]] and is right *)
(* not
to. *)
ClearAll[cfSignatureAssume];
cfSignatureAssume = cfGeomAssume && Element[a4[H x4], Reals] && 0 < 6 H x0 < Pi/2;
cfAssert[

```

```

"CANONICAL [has content]: g has signature (4,4), GIVEN a4 real and  $0 < \theta < \pi/2$ ",
Module[{s = Simplify[# > 0, cfSignatureAssume] & /@ Diagonal[gCanonical]},
  {Count[s, True] == 4, Count[s, False] == 4}]];
cfAssert["Inverse[g] == Transpose[coframe] . eta4488 . coframe",
  cfZeroArrayQ[gInvCanonical - Transpose[coframeCanonical] .  $\eta_{4488}$  . coframeCanonical]];
cfAssert["g is symmetric", cfZeroArrayQ[gCanonical - Transpose[gCanonical]]];
MatrixForm[gCanonical]

(* --- the metric determinant,
and the volume factor used in Lagrangians ----- *)
ClearAll[detgCanonical, sqrtDetgCanonical];
detgCanonical = cfSimp[Det[gCanonical]];
sqrtDetgCanonical = cfSimp[Sqrt[Abs[detgCanonical]]];
cfAssert["the metric is non-degenerate", ! TrueQ[cfZeroQ[detgCanonical]]];
{detgCanonical, sqrtDetgCanonical}

(* --- the frame, coframe and metric,
direction by direction ----- *)
Grid[Prepend[
  Table[{Subscript["direction", ToString[ $\mu$  - 1]],
    frameCanonical[[ $\mu$ ,  $\mu$ ]],
    coframeCanonical[[ $\mu$ ,  $\mu$ ]],
    gCanonical[[ $\mu$ ,  $\mu$ ]], { $\mu$ , 1, 8}},
{"direction", "frame e[ $\mu$ ,a]", "coframe e_a^ $\mu$ ", "g[ $\mu$ , $\mu$ "]}],
Frame → All, Alignment → Left]

```

16. The generalized Christoffel symbols, and the canonical spin connection

Step 2 of the standard procedure. The spin connection is obtained from the zero-torsion (Levi-Civita) condition in the form known as the VIELBEIN POSTULATE: the total covariant derivative of the frame field, using the affine connection on the curved index and the spin connection on the flat index, vanishes:

$$D[e[\nu, a], x[\mu]] - \Gamma[\rho, \mu, \nu] e[\rho, a] + \omega_{\text{Mixed}}[\mu, a, b] e[\nu, b] == 0.$$

Solving it for ω , and contracting with the coframe to free the index ν ,

$$\omega_{\text{Mixed}}[\mu, a, c] = \text{Sum over } \nu \text{ of} \\ (\text{Sum over } \rho \text{ of } \Gamma[\rho, \mu, \nu] e[\rho, a] - D[e[\nu, a], x[\mu]]) * \text{coframe}[[c, \nu]].$$

Lowering the first flat index with η gives $\omega[\mu, a, b]$, which must come out ANTISYMMETRIC in a and b . That antisymmetry is not imposed: it is a consequence of metric compatibility, and we verify it explicitly for every connection computed in this notebook.

```

(* --- the generalized Christoffel symbols
of the second kind ----- *)
(* Gamma[[rho,mu,nu]] =
(1/2) g^{rho s} ( d_mu g_{s nu} + d_nu g_{s mu} - d_s g_{mu nu} ) *)
(* Optimization:
the first derivatives of g are computed once into dg[[k,m,p]] and reused, *)
(* instead of being recomputed inside the
triple loop as a naive transcription would do. *)
ClearAll[cfChristoffel];
cfChristoffel[g_, ginv_, coords_] := Module[{n = Length[coords], dg},
dg = Table[D[g[[m, p]], coords[[k]], {k, n}, {m, n}, {p, n}];
Table[
cfSimp[(1/2) Sum[ginv[[r, s]] (dg[[m, s, p]] + dg[[p, s, m]] - dg[[s, m, p]]), {s, n}]],
{r, n}, {m, n}, {p, n}];
ClearAll[GammaCanonical];
GammaCanonical = cfTimed["generalized Christoffel symbols of the 4+4 metric",
cfChristoffel[gCanonical, gInvCanonical, X]];
cfAssert[
"Gamma is symmetric in its two lower indices (torsion-free affine connection)",
cfZeroArrayQ[Table[GammaCanonical[[r, m, p]] - GammaCanonical[[r, p, m]],
{r, 8}, {m, 8}, {p, 8}]]];
{Dimensions[GammaCanonical],
Count[Flatten[GammaCanonical], Except[0]], " non-zero components"}

(* --- the non-zero Christoffel symbols,
listed ----- *)
Grid[Prepend[
Select[Flatten[Table[{Row[{"Gamma[" , r - 1, ";", m - 1, ",", p - 1, ""]}],
GammaCanonical[[r, m, p]], {r, 8}, {m, 8}, {p, 8}], 2], #[[2]] != 0 &],
{"symbol", "value"}], Frame -> All, Alignment -> Left]

```

```

(* =====
===== *)
(* The general spin-
connection solver. It takes ANY frame field and ANY flat tangent metric *)
(* and returns the unique zero-
torsion spin connection determined by the vielbein postulate. *)
(* Every one of the four connections in this
notebook is produced by this one routine, so *)
(* the comparisons of Part IV compare results,
not different pieces of code. *)
(* The Christoffel symbols are a property of g alone,
so they are passed in and REUSED *)
(* across all four bridges rather than
recomputed four times. *)
(* =====
===== *)
ClearAll[cfSpinConnection, cfLowerFirstFlat,
cfVielbeinResidual, cfTorsion, cfCurvature];
cfSpinConnection[frame_, coords_, etaFlat_, gamma_] :=
Module[{n = Length[coords], cofr, dE},
cofr = cfSimp[Inverse[frame]]; (* cofr[[a,nu]] = e_a^nu *)
dE = Table[D[frame[[nu, a]], coords[[mu]]], {mu, n}, {nu, n}, {a, n}];
Table[
cfSimp[Sum[(Sum[gamma[[r, mu, nu]] frame[[r, a]], {r, n}] - dE[[mu, nu, a]]) cofr[[c, nu]],
{nu, n}]],
{mu, n}, {a, n}, {c, n}]];

```

```

(* --- index lowering,
and the three diagnostics every connection must pass ----- *)
cfLowerFirstFlat[om_, etaFlat_] := Module[{n = Length[etaFlat]},
  Table[Sum[etaFlat[[a, c]] om[[mu, c, b]], {c, n}], {mu, n}, {a, n}, {b, n}];
cfVielbeinResidual[frame_, coords_, gamma_, om_] := Module[{n = Length[coords]},
  Table[
    D[frame[[nu, a]], coords[[mu]]] - Sum[gamma[[r, mu, nu]] frame[[r, a]], {r, n}]
    + Sum[om[[mu, a, b]] frame[[nu, b]], {b, n}],
    {mu, n}, {nu, n}, {a, n}];
(* torsion 2-form  $T^a_{\mu\nu}$  =
 $d_\mu e_\nu^a - d_\nu e_\mu^a + \omega_{\mu\nu}^{\phantom{\mu\nu}a}{}_b e_\nu^b - (\mu \leftrightarrow \nu)$  *)
cfTorsion[frame_, coords_, om_] := Module[{n = Length[coords]},
  cfSimpArray @ Table[
    D[frame[[nu, a]], coords[[mu]]] - D[frame[[mu, a]], coords[[nu]]
    + Sum[om[[mu, a, b]] frame[[nu, b]] - om[[nu, a, b]] frame[[mu, b]], {b, n}],
    {a, n}, {mu, n}, {nu, n}];
(* curvature 2-form  $R_{\mu\nu}^{\phantom{\mu\nu}a}{}_b =
 $d_\mu \omega_{\nu}^a - d_\nu \omega_\mu^a + [\omega_\mu, \omega_\nu]^a{}_b$  *)
cfCurvature[om_, coords_] := Module[{n = Length[coords]},
  cfSimpArray @ Table[
    D[om[[nu, a, b]], coords[[mu]]] - D[om[[mu, a, b]], coords[[nu]]
    + Sum[om[[mu, a, c]] om[[nu, c, b]] - om[[nu, a, c]] om[[mu, c, b]], {c, n}],
    {mu, n}, {nu, n}, {a, n}, {b, n}];

(* =====
THE CANONICAL SPIN CONNECTION ===== *)
ClearAll[omegaCanonicalMixed, omegaCanonical];
omegaCanonicalMixed =
  cfTimed["canonical spin connection omega[mu]^a_b from the vielbein postulate",
  cfSpinConnection[frameCanonical, X,  $\eta_{4488}$ , GammaCanonical]];
omegaCanonical = cfLowerFirstFlat[omegaCanonicalMixed,  $\eta_{4488}$ ];
{Dimensions[omegaCanonical],
  Count[Flatten[omegaCanonical], Except[0]], " non-zero components"}$ 
```

Now the verification, and it is worth being exact about what each check does and does not establish, because two of the three obvious ones are weaker than they look.

- "the vielbein postulate residual is zero" is an ALGEBRAIC IDENTITY of the solver, not a fact about this geometry. cfSpinConnection contracts the postulate with Inverse[frame], and cfVielbeinResidual contracts it back with frame; the two contractions cancel for ANY affine connection and ANY invertible frame. It is a useful regression test of the code, and nothing more.
- "the torsion vanishes" is nearly as weak. Once the residual is zero, the torsion reduces to $(\Gamma[\rho, \mu, \nu] - \Gamma[\rho, \nu, \mu]) e[\rho, a]$, and the symmetry of Gamma in its lower indices is itself structural: swapping mu and nu in $(1/2) g^{\{rs\}}(dg[m, s, p] + dg[p, s, m] - dg[s, m, p])$ reproduces the same expression whenever g is symmetric.

- " $\omega[\mu, a, b] == -\omega[\mu, b, a]$ " DOES have content. Nothing in the solver imposes it; it holds only because the affine connection fed in really is the Levi-Civita connection of the metric that the frame builds. A wrong Christoffel symbol would break it.

So we run all three, labelled honestly, and then add a fourth check that is genuinely independent: re-derive the spin connection from the FRAME ALONE, by the standard closed formula in terms of the anholonomy of the coframe, which never mentions Gamma or the metric, and compare. If cfChristoffel or cfSpinConnection had an index slot wrong, that comparison would fail.

```
(* --- what the three obvious checks actually
  establish ----- *)
cfAssert["CANONICAL [solver regression]: vielbein postulate residual is zero",
  cfZeroArrayQ[
    cfVielbeinResidual[frameCanonical, X, GammaCanonical, omegaCanonicalMixed]]];
cfAssert["CANONICAL [structural]: torsion vanishes",
  cfZeroArrayQ[cfTorsion[frameCanonical, X, omegaCanonicalMixed]]];
cfAssert[
  "CANONICAL [has content]:  $\omega[\mu, a, b] == -\omega[\mu, b, a]$  (metric compatibility)",
  cfZeroArrayQ[Table[omegaCanonical[[mu, a, b]] + omegaCanonical[[mu, b, a]],
    {mu, 8}, {a, 8}, {b, 8}]]];
```

```

(* --- the independent derivation,
from the frame alone ----- *)
(* omega_mu^{ab} =
(1/2) e^{a nu} ( d_mu e_nu^b - d_nu e_mu^b ) *)
(*
- (1/2) e^{b nu}
( d_mu e_nu^a - d_nu e_mu^a ) *)
(*
- (1/2) e^{a rho} e^{b sig}
( d_rho e_sig^c - d_sig e_rho^c ) eta_{cd} e_mu^d *)
(* Nothing here refers to Gamma or to g: only the frame,
its derivatives and eta. *)
ClearAll[cfSpinConnectionFromFrame];
cfSpinConnectionFromFrame[frame_, coords_, etaFlat_] :=
Module[{n = Length[coords], cofr, up, dE, term1, term2, term3},
cofr = cfSimp[Inverse[frame]]; (* cofr[[a,nu]] = e_a^nu *)
up = cfSimp[Inverse[etaFlat] . cofr]; (* up[[a,nu]] = e^{a nu} *)
dE = Table[D[frame[[nu, a]], coords[[mu]], {mu, n}, {nu, n}, {a, n}];
Table[
cfSimp[
(1/2) Sum[up[[a, nu]] (dE[[mu, nu, b]] - dE[[nu, mu, b]]), {nu, n}]
- (1/2) Sum[up[[b, nu]] (dE[[mu, nu, a]] - dE[[nu, mu, a]]), {nu, n}]
- (1/2) Sum[up[[a, rho]] up[[b, sig]] (dE[[rho, sig, c]] - dE[[sig, rho, c]]) ×
etaFlat[[c, d]] frame[[mu, d]], {rho, n}, {sig, n}, {c, n}, {d, n}]],
{mu, n}, {a, n}, {b, n}]];

ClearAll[omegaFromFrameUp];
omegaFromFrameUp = cfTimed["independent derivation of omega from the frame alone",
cfSpinConnectionFromFrame[frameCanonical, X, η4488]];
(* omegaFromFrameUp carries omega_mu^{ab}; lower the second flat index to compare *)
cfAssert[
"INDEPENDENT CHECK: the frame-only derivation reproduces omegaCanonical exactly",
cfZeroArrayQ[Table[Sum[omegaFromFrameUp[[mu, a, c]] η4488[[c, b]], {c, 8}]
- omegaCanonicalMixed[[mu, a, b]], {mu, 8}, {a, 8}, {b, 8}]]];
cfAssert["INDEPENDENT CHECK: and it is antisymmetric in its two raised flat indices",
cfZeroArrayQ[Table[omegaFromFrameUp[[mu, a, b]] + omegaFromFrameUp[[mu, b, a]],
{mu, 8}, {a, 8}, {b, 8}]]];

(* --- the non-zero components of the canonical spin connection,
listed ----- *)
ClearAll[cfShowConnection];
cfShowConnection[om_, name_String] := Grid[Prepend[
Select[Flatten[Table[{Row[{name, "[", mu - 1, ";", a - 1, ", ", b - 1, "]" }],
om[[mu, a, b]], {mu, 8}, {a, 8}, {b, 8}], 2], #[[2]] != 0 &],
{"component", "value"}], Frame → All, Alignment → Left];
cfShowConnection[omegaCanonical, "omega"]

```

```

(* --- the curvature 2-
form of the canonical connection ----- *)
ClearAll[RiemannCanonical];
RiemannCanonical = cfTimed["curvature 2-form of the canonical spin connection",
cfCurvature[omegaCanonicalMixed, X]];
cfAssert["CANONICAL: curvature is antisymmetric in the two spacetime indices",
cfZeroArrayQ[Table[RiemannCanonical[[mu, nu, a, b]] + RiemannCanonical[[nu, mu, a, b]],
{mu, 8}, {nu, 8}, {a, 8}, {b, 8}]]];
{Count[Flatten[RiemannCanonical], Except[0]], " non-zero curvature components"}

(* --- the Ricci scalar;
it is the invariant we use to confirm that every bridge below ----- *)
(* --- really does describe the SAME
geometry ----- *)
ClearAll[cfRicciScalar, RicciScalarCanonical];
cfRicciScalar[frame_, riem_, ginv_] := Module[{cofr, ric},
cofr = cfSimp[Inverse[frame]];
ric = cfSimpArray @ Table[Sum[cofr[[a, mu]] riem[[mu, nu, a, b]], {a, 8}, {mu, 8}],
{nu, 8}, {b, 8}];
(* one scalar,
so FullSimplify is cheap here and it matters: without it the same number *)
(* comes out in different surface forms from different frames,
which looks like a *)
(* disagreement when it is
not. *)
FullSimplify[Sum[ginv[[nu, rho]] frame[[rho, b]] ric[[nu, b]], {nu, 8}, {rho, 8}, {b, 8}],
cfGeomAssume]];
RicciScalarCanonical = cfTimed["Ricci scalar of the canonical connection",
cfRicciScalar[frameCanonical, RiemannCanonical, gInvCanonical]];
RicciScalarCanonical

```

17. The gauge-covariant derivative of the 16-component spinor

Step 3 of the standard procedure. The wave function of this model is a 16-component spinor that is the DIRECT SUM of a split-octonion type-1 spinor field and a split-octonion type-2 spinor field:

$$\text{Psi}_{16} = \{ \text{psi}_1 \text{ (8 components)} , \text{psi}_2 \text{ (8 components)} \}.$$

Acting on it, ordinary partial derivatives are replaced by the gauge-covariant derivative

$$\text{Dcov}[\mu] \text{Psi} == \text{D}[\text{Psi}, \text{x}[\mu]] + (1/8) \text{omega}[\mu, \text{a}, \text{b}] \text{Commutator}[\text{gamma}[\text{a}], \text{gamma}[\text{b}]] . \text{Psi}$$

where $\gamma[a] = T16[a]$ are the 16x16 Dirac matrices of Section 5. Since $SAB[[a+1,b+1]] = (1/4) \text{Commutator}[\gamma[a], \gamma[b]]$, the connection term is equally $(1/2) \omega[\mu,a,b] SAB[[a+1,b+1]]$, which is the form the original notebook's commented-out Lagrangian Lg uses.

A structural fact that matters for this model: each $\gamma[a]$ is OFF-diagonal in the type-1/type-2 block decomposition, so every PRODUCT of two gammas is BLOCK-DIAGONAL. The spin connection matrix therefore never mixes the type-1 spinor with the type-2 spinor. The 16-component covariant derivative really is the direct sum of a type-1 covariant derivative and a type-2 covariant derivative, and we verify that below and extract the two 8x8 blocks.

```
(* --- the 16x16 spin-
connection matrix in each spacetime direction ----- *)
ClearAll[cfSpinMatrix, GammaSpinCanonical];
cfSpinMatrix[om_, mu_Integer] :=
  (1/8)
  Sum[om[[mu, a, b]] (T16[a - 1] . T16[b - 1] - T16[b - 1] . T16[a - 1]), {a, 8}, {b, 8}];
GammaSpinCanonical = cfTimed["16x16 spin-connection matrices Gamma^spin[mu]",
  Table[cfSimpArray[cfSpinMatrix[omegaCanonical, mu]], {mu, 8}]];
Dimensions[GammaSpinCanonical]

(* --- it equals (1/2) omega[mu,a,b] SAB,
as it must ----- *)
cfAssert["(1/8) omega [gamma^a, gamma^b] == (1/2) omega SAB",
  cfZeroArrayQ[Table[GammaSpinCanonical[[mu]]
    - (1/2) Sum[omegaCanonical[[mu, a, b]] SAB[a, b], {a, 8}, {b, 8}], {mu, 8}]]];

(* --- the direct-
sum structure: every Gamma^spin[mu] is block diagonal ----- *)
ClearAll[GammaSpinType1, GammaSpinType2];
cfAssert["Gamma^spin[mu] is block diagonal: it never mixes type-1 with type-2",
  cfZeroArrayQ[Table[GammaSpinCanonical[[mu]]
    - ArrayFlatten[{{GammaSpinCanonical[[mu]][[1 ;; 8, 1 ;; 8]], 0},
      {0, GammaSpinCanonical[[mu]][[9 ;; 16, 9 ;; 16]]}], {mu, 8}]]];
GammaSpinType1 = Table[GammaSpinCanonical[[mu]][[1 ;; 8, 1 ;; 8]], {mu, 8}];
GammaSpinType2 = Table[GammaSpinCanonical[[mu]][[9 ;; 16, 9 ;; 16]], {mu, 8}];
{Dimensions[GammaSpinType1], Dimensions[GammaSpinType2]}
```

```

(* --- the two 8x8 blocks,
written directly in terms of tau and taubar ----- *)
cfAssert["the type-1 block is (1/8) omega (taubar^a tau^b - taubar^b tau^a)",
cfZeroArrayQ[Table[GammaSpinType1[[mu]] - (1/8) Sum[omegaCanonical[[mu, a, b]] ×
  (τbar[a - 1] . τ[b - 1] - τbar[b - 1] . τ[a - 1]), {a, 8}, {b, 8}],
  {mu, 8}]]];
cfAssert["the type-2 block is (1/8) omega (tau^a taubar^b - tau^b taubar^a)",
cfZeroArrayQ[Table[GammaSpinType2[[mu]] - (1/8) Sum[omegaCanonical[[mu, a, b]] ×
  (τ[a - 1] . τbar[b - 1] - τ[b - 1] . τbar[a - 1]), {a, 8}, {b, 8}],
  {mu, 8}]]];

(* --- the covariant derivative
  itself ----- *)
(* cfDcov16 acts on a 16-component spinor;
cfDcovType1 and cfDcovType2 act on the type-1 and *)
(* type-2 split-octonion spinors separately,
which is legitimate exactly because the *)
(* connection matrix is block
  diagonal. *)
ClearAll[cfDcov16, cfDcovType1, cfDcovType2, psiGeneric, psi1Generic, psi2Generic];
cfDcov16[psi_List, mu_Integer, gs_ : Automatic] :=
  D[psi, X[[mu]]] + (If[gs == Automatic, GammaSpinCanonical, gs] [[mu]]).psi;
cfDcovType1[psi1_List, mu_Integer] := D[psi1, X[[mu]]] + GammaSpinType1[[mu]].psi1;
cfDcovType2[psi2_List, mu_Integer] := D[psi2, X[[mu]]] + GammaSpinType2[[mu]].psi2;
psiGeneric = Table[cfPsi[k] @@ X, {k, 0, 15}];
psi1Generic = psiGeneric[[1 ;; 8]];
psi2Generic = psiGeneric[[9 ;; 16]];
cfAssert["Dcov16 on the direct sum is the direct sum of Dcov1 and Dcov2",
cfZeroArrayQ[Table[cfDcov16[psiGeneric, mu]
  - Join[cfDcovType1[psi1Generic, mu], cfDcovType2[psi2Generic, mu]], {mu, 8}]]];
cfDcovType1[psi1Generic, 1] // Short[#, 6] &

(* --- the curved-
space Dirac operator ----- *)
(* gammaCurved[mu] =
coframe[[a,mu]] gamma^a obeys {gammaCurved[mu], gammaCurved[nu]} *)
(*
  2 g^{mu nu} ID16 . *)
ClearAll[gammaCurvedCanonical, cfDiracOperator];
gammaCurvedCanonical =
  Table[cfSimpArray[Sum[coframeCanonical[[a, mu]] T16[a - 1], {a, 8}], {mu, 8}];
cfAssert["{gammaCurved[mu], gammaCurved[nu]} == 2 gInv[mu,nu] ID16",
cfZeroArrayQ[Table[gammaCurvedCanonical[[mu]] . gammaCurvedCanonical[[nu]]
  + gammaCurvedCanonical[[nu]] . gammaCurvedCanonical[[mu]]
  - 2 gInvCanonical[[mu, nu]] ID16, {mu, 8}, {nu, 8}]]];
cfDiracOperator[psi_List] :=
  Sum[gammaCurvedCanonical[[mu]] . cfDcov16[psi, mu], {mu, 8}];
Dimensions[gammaCurvedCanonical]

```

PART IV -- Three new bridges between curved and flat indices, and three new spin connections

The canonical frame field of Part III is one choice among infinitely many. We now invent three genuinely different ways of bridging curved spacetime indices to flat tangent-space indices, derive a new spin connection for each one with the SAME solver `cfSpinConnection`, and compare each of them, component by component, with the canonical spin connection.

The three bridges are of three different kinds, so the comparisons are of three different kinds:

BRIDGE 1 a locally boosted orthonormal frame. The flat metric is still `eta4488` but the frame is rotated at every point by an x -dependent element of $O(4,4)$. The new connection differs from the canonical one by an INHOMOGENEOUS gauge term. This exhibits the spin connection as the gauge field of local Lorentz transformations.

BRIDGE 2 a null (light-cone) frame. The flat tangent metric is CHANGED, from `eta4488` to the split form in which the four pairs (x_0, x_4) , (x_1, x_5) , (x_2, x_6) , (x_3, x_7) are null. That split form turns out to be exactly the split-octonion spinor metric `sigma` of Section 4. The new connection differs from the canonical one by a CONSTANT similarity, with no inhomogeneous term. This is the frame adapted to the Einstein-Rosen pairing of the two sheets.

BRIDGE 3 a triality (split-octonion spinor) frame carrying TOTALLY ANTISYMMETRIC TORSION built from the split-octonion structure constants. The flat index is a type-1 spinor index, and the connection is NOT the Levi-Civita one: it differs from it by a contortion term, so its torsion is non-zero. This is the only one of the three that is a different CONNECTION rather than a different description of the same one.

A general fact we will use three times. If a new frame is built from the old one by $\text{frameNew} = \text{frameOld} \cdot L$ with L an invertible matrix of flat indices, then substituting into the vielbein postulate and setting $M = \text{Transpose}[L]$ gives

$$\omega_{\text{New}}[\mu] == M \cdot \omega_{\text{Old}}[\mu] \cdot \text{Inverse}[M] - D[M, x_{\mu}] \cdot \text{Inverse}[M] \cdot$$

The second term is the inhomogeneous gauge term; it vanishes exactly when L is constant. We do

NOT assume this formula anywhere: each new connection is derived independently from the vielbein postulate, and the formula is then verified against the derived result.

18. Bridge 1 -- a locally boosted orthonormal frame, and the gauge nature of omega

Introduce a NEW object, not present in the original notebook: a rapidity field $\text{cfBoostRapidity}[x_0, x_4]$, an arbitrary differentiable scalar function of the hidden-space coordinate x_0 and the time coordinate x_4 . It generates a boost in the flat (0,4) plane, i.e. between the first (spacelike, $\eta = +1$) and the fifth (timelike, $\eta = -1$) tangent directions:

$$\begin{aligned}\text{Lambda}[\text{theta}] &= \text{ID8} + (\text{Cosh}[\text{theta}] - 1)(\text{E00} + \text{E44}) + \text{Sinh}[\text{theta}](\text{E04} + \text{E40}) \\ &= \text{MatrixExp}[\text{theta K}], \quad \text{K the boost generator on flat indices 0 and 4.}\end{aligned}$$

Lambda is an element of $O(4,4)$: $\text{Lambda} \cdot \eta_{4488} \cdot \text{Transpose}[\text{Lambda}] = \eta_{4488}$. The new frame field is $\text{frameBoost} = \text{frameCanonical} \cdot \text{Lambda}$. Because Lambda preserves η_{4488} , THE CURVED METRIC IS UNCHANGED: the two frames describe the same geometry through two different local Minkowski coordinate systems, related by a different boost at every point.

```
ClearAll[cfBoostRapidity, cfLambda, cfLambdaOf, cfBoostGenerator, frameBoost];
cfBoostGenerator = Module[{K = ConstantArray[0, {8, 8}]}, K[[1, 5]] = 1; K[[5, 1]] = 1; K];
cfLambdaOf[th_] := Module[{L = IdentityMatrix[8]},
  L[[1, 1]] = Cosh[th]; L[[5, 5]] = Cosh[th];
  L[[1, 5]] = Sinh[th]; L[[5, 1]] = Sinh[th];
  L];
cfLambda = cfLambdaOf[cfBoostRapidity[x0, x4]];
cfAssert["Lambda is in O(4,4): Lambda . eta . Transpose[Lambda] == eta",
  cfZeroArrayQ[cfLambda .  $\eta_{4488}$  . Transpose[cfLambda] -  $\eta_{4488}$ ]];
cfAssert["Lambda is NOT in O(8): it is a genuine boost, not a rotation",
  ! TrueQ[cfZeroArrayQ[cfLambda . Transpose[cfLambda] - ID8]]];
cfAssert["Lambda == MatrixExp[theta K] with K the (0,4) boost generator",
  cfZeroArrayQ[cfLambdaOf[0] - MatrixExp[0 cfBoostGenerator]]];
MatrixForm[cfLambda]

frameBoost = cfSimpArray[frameCanonical . cfLambda];
cfAssert["BRIDGE 1 reproduces the SAME curved metric g",
  cfZeroArrayQ[frameBoost .  $\eta_{4488}$  . Transpose[frameBoost] - gCanonical]];
cfAssert["BRIDGE 1 frame really is different from the canonical frame",
  frameBoost != frameCanonical];
MatrixForm[frameBoost]
```

```

(* --- the new spin connection,
derived from the same vielbein postulate ----- *)
ClearAll[omegaBoostMixed, omegaBoost];
omegaBoostMixed = cfTimed["BRIDGE 1 spin connection",
  cfSpinConnection[frameBoost, X,  $\eta_{4488}$ , GammaCanonical]]];
omegaBoost = cfLowerFirstFlat[omegaBoostMixed,  $\eta_{4488}$ ];
cfAssert["BRIDGE 1: vielbein postulate holds identically",
  cfZeroArrayQ[cfVielbeinResidual[frameBoost, X, GammaCanonical, omegaBoostMixed]]];
cfAssert["BRIDGE 1: omega[mu,a,b] == -omega[mu,b,a]",
  cfZeroArrayQ[
    Table[omegaBoost[[mu, a, b]] + omegaBoost[[mu, b, a]], {mu, 8}, {a, 8}, {b, 8}]]];
cfAssert["BRIDGE 1: torsion still vanishes",
  cfZeroArrayQ[cfTorsion[frameBoost, X, omegaBoostMixed]]];
{Count[Flatten[omegaBoost], Except[0]], " non-zero components"}

(* --- COMPARISON 1:
the new connection is the canonical one plus a pure gauge term ----- *)
ClearAll[cfM, cfMinv, omegaBoostPredicted, deltaBoost, gaugeTermBoost];
cfM = Transpose[cfLambda];
cfMinv = cfSimpArray[Inverse[cfM]];
omegaBoostPredicted = Table[
  cfSimpArray[cfM.omegaCanonicalMixed[[mu]].cfMinv - D[cfM, X[[mu]]].cfMinv], {mu, 8}];
cfAssert[
  "COMPARISON 1: omegaBoost == M omegaCanonical Inverse[M] - D[M,x_mu] Inverse[M]",
  cfZeroArrayQ[omegaBoostMixed - omegaBoostPredicted]];
gaugeTermBoost = Table[cfSimpArray[-D[cfM, X[[mu]]].cfMinv], {mu, 8}];
deltaBoost =
  Table[cfSimpArray[omegaBoostMixed[[mu]] - cfM.omegaCanonicalMixed[[mu]].cfMinv],
    {mu, 8}];
cfAssert["COMPARISON 1: the WHOLE difference is the inhomogeneous gauge term",
  cfZeroArrayQ[deltaBoost - gaugeTermBoost]];
{Count[Flatten[deltaBoost], Except[0]], " non-zero components in the difference"}

```

```

(* --- the gauge term written
out: it is -d(rapidity) times the boost generator ----- *)
(* Because Lambda == MatrixExp[theta K],
D[Lambda,x_mu] . Inverse[Lambda] == D[theta,x_mu] K *)
(* exactly, with no residual theta
dependence. The whole difference between the two spin *)
(* connections is therefore one scalar
gradient times one fixed generator. *)
cfAssert["COMPARISON 1: the difference equals -D[theta, x_mu] * (boost generator)",
cfZeroArrayQ[Table[deltaBoost[[mu]]
+ D[cfBoostRapidity[x0, x4], X[[mu]]] cfBoostGenerator, {mu, 8}]]];
Grid[Prepend[
Table[{Row[{"Delta1[" , mu - 1, ""]}], -D[cfBoostRapidity[x0, x4], X[[mu]]],
If[D[cfBoostRapidity[x0, x4], X[[mu]]] === 0, "zero",
"times the (0,4) boost generator K"]}],
{mu, 8}], {"direction mu", "coefficient", "structure"}],
Frame -> All, Alignment -> Left]

(* --- and the curvature is only conjugated: same geometry,
new gauge ----- *)
ClearAll[RiemannBoost, RicciScalarBoost];
RiemannBoost = cfTimed["BRIDGE 1 curvature 2-form", cfCurvature[omegaBoostMixed, X]];
cfAssert["COMPARISON 1: curvature transforms covariantly, R' == M R Inverse[M]",
cfZeroArrayQ[Table[RiemannBoost[[mu, nu]] - cfM . RiemannCanonical[[mu, nu]] . cfMinv,
{mu, 8}, {nu, 8}]]];
RicciScalarBoost = cfTimed["BRIDGE 1 Ricci scalar",
cfRicciScalar[frameBoost, RiemannBoost, gInvCanonical]];
cfAssert["COMPARISON 1: BRIDGE 1 has the SAME Ricci scalar as the canonical frame",
cfZeroQ[RicciScalarBoost - RicciScalarCanonical]];
{RicciScalarCanonical, RicciScalarBoost}

```

19. Bridge 2 -- the null (light-cone) frame, and the appearance of the spinor metric sigma

The second bridge changes the FLAT TANGENT METRIC rather than the frame's orientation. Pair the eight flat directions as (0,4), (1,5), (2,6), (3,7) -- one spacelike with one timelike in each pair -- and replace each pair by its two null combinations. The constant matrix that does this is

$$U = (1/\text{Sqrt}[2]) \text{ArrayFlatten}[\{\{ID4, ID4\}, \{ID4, -ID4\}\}], \quad U == \text{Transpose}[U], \\ U . U == ID8 .$$

Under U the flat metric becomes $\text{etaNull} = \text{Transpose}[U] . \text{eta4488} . U$, and the remarkable fact, verified below, is that etaNull IS EXACTLY THE SPLIT-OCTONION SPINOR METRIC sigma of Section 4.

The null frame of this 4+4 geometry and the spinor pairing of the split octonions are the same structure. Note also that η_{Null} has a zero diagonal, which is precisely the statement that all eight frame directions are null.

This is the frame adapted to Einstein and Rosen's picture: each pair of coordinates is split into an advanced and a retarded null direction, which is exactly the (ztadv, ztret) and (xiadv, xiret) splitting the original notebook performs by hand in its "retarded, advanced" sections.

The new frame is $\text{frameNull} = \text{frameCanonical} \cdot U$, and it reproduces the same curved metric g because $U \cdot \eta_{\text{Null}} \cdot \text{Transpose}[U] == \eta_{4488}$.

```
ClearAll[cfU,  $\eta_{\text{Null}}$ , frameNull];
cfU = (1/Sqrt[2]) ArrayFlatten[{{ID4, ID4}, {ID4, -ID4}}];
cfAssert["U is an involution:  $U \cdot U == \text{ID8}$ ", cfZeroArrayQ[cfU.cfU - ID8]];
cfAssert["U is symmetric", cfU === Transpose[cfU]];
 $\eta_{\text{Null}}$  = FullSimplify[Transpose[cfU]. $\eta_{4488}$ .cfU];
cfAssert["THE NULL TANGENT METRIC IS EXACTLY THE SPINOR METRIC  $\sigma$ ",  $\eta_{\text{Null}}$  ===  $\sigma$ ];
cfAssert[" $\eta_{\text{Null}}$  is symmetric and squares to ID8",
  { $\eta_{\text{Null}}$  === Transpose[ $\eta_{\text{Null}}$ ],  $\eta_{\text{Null}} \cdot \eta_{\text{Null}}$  === ID8}];
cfAssert["all eight frame directions are null:  $\eta_{\text{Null}}$  has zero diagonal",
  Diagonal[ $\eta_{\text{Null}}$ ] === ConstantArray[0, 8]];
MatrixForm[ $\eta_{\text{Null}}$ ]

frameNull = cfSimpArray[frameCanonical.cfU];
cfAssert["BRIDGE 2 reproduces the SAME curved metric  $g$ ",
  cfZeroArrayQ[frameNull. $\eta_{\text{Null}}$ .Transpose[frameNull] - gCanonical]];
MatrixForm[frameNull]

(* --- the new spin
   connection ----- *)
ClearAll[omegaNullMixed, omegaNull];
omegaNullMixed = cfTimed["BRIDGE 2 spin connection",
  cfSpinConnection[frameNull, X,  $\eta_{\text{Null}}$ , GammaCanonical]];
omegaNull = cfLowerFirstFlat[omegaNullMixed,  $\eta_{\text{Null}}$ ];
cfAssert["BRIDGE 2: vielbein postulate holds identically",
  cfZeroArrayQ[cfVielbeinResidual[frameNull, X, GammaCanonical, omegaNullMixed]]];
cfAssert["BRIDGE 2:  $\omega_{[\mu,a,b]} == -\omega_{[\mu,b,a]}$  with indices lowered by  $\eta_{\text{Null}}$ ",
  cfZeroArrayQ[
    Table[omegaNull[[ $\mu$ , a, b]] + omegaNull[[ $\mu$ , b, a]], { $\mu$ , 8}, {a, 8}, {b, 8}]]];
cfAssert["BRIDGE 2: torsion vanishes",
  cfZeroArrayQ[cfTorsion[frameNull, X, omegaNullMixed]]];
{Count[Flatten[omegaNull], Except[0]], " non-zero components"}
```

```

(* --- COMPARISON 2: a CONSTANT similarity,
with no inhomogeneous term ----- *)
(* Here L == U, so M == Transpose[U] == U and Inverse[M] == U;
and because U is constant the *)
(* gauge term D[M,x_mu] . Inverse[M] is
identically zero. *)
ClearAll[deltaNull];
cfAssert["COMPARISON 2: omegaNull == U . omegaCanonical . U exactly",

cfZeroArrayQ[Table[omegaNullMixed[[mu]] - cfU . omegaCanonicalMixed[[mu]] . cfU, {mu, 8}]]];
deltaNull = cfSimpArray[Table[omegaNullMixed[[mu]] - cfU . omegaCanonicalMixed[[mu]] . cfU,
{mu, 8}]]];
cfAssert["COMPARISON 2: the inhomogeneous gauge term is ZERO because U is constant",
cfZeroArrayQ[Table[D[cfU, X[[mu]]], {mu, 8}]]];
cfShowConnection[omegaNull, "omegaNull"]

(* --- the curvature is the constant conjugate,
so again the same geometry ----- *)
ClearAll[RiemannNull, RicciScalarNull];
RiemannNull = cfTimed["BRIDGE 2 curvature 2-form", cfCurvature[omegaNullMixed, X]];
cfAssert["COMPARISON 2: R_null == U . R_canonical . U",
cfZeroArrayQ[Table[RiemannNull[[mu, nu]] - cfU . RiemannCanonical[[mu, nu]] . cfU,
{mu, 8}, {nu, 8}]]];
RicciScalarNull = cfTimed["BRIDGE 2 Ricci scalar",
cfRicciScalar[frameNull, RiemannNull, gInvCanonical]];
cfAssert["COMPARISON 2: BRIDGE 2 has the SAME Ricci scalar as the canonical frame",
cfZeroQ[RicciScalarNull - RicciScalarCanonical]];
{RicciScalarCanonical, RicciScalarNull}

```

20. Bridge 3 -- the triality frame with totally antisymmetric split-octonion torsion

The third bridge is of a different kind. Its flat index is not a vector index at all: it is a TYPE-1 SPLIT-OCTONION SPINOR INDEX, reached through the constant triality bridge `triVecToSpin` of Section 9. Section 9 proved that this bridge carries `eta4488` exactly onto `sigma`, so the frame

```
frameTri = frameCanonical . triVecToSpin
```

again reproduces the same curved metric, now with flat tangent metric `sigma`. We also record here a fact that is used below and is worth stating on its own: `triVecToSpin` is an ORTHOGONAL matrix, `Transpose[triVecToSpin] == Inverse[triVecToSpin]`. The triality bridge is a rotation of the 8-dimensional flat space, not merely a change of basis.

That alone would give nothing new beyond Bridge 2, because `triVecToSpin` is constant. So we go

further and change the CONNECTION as well. The split-octonion structure constants m , restricted to the seven imaginary directions and with all indices lowered, are TOTALLY ANTISYMMETRIC (this was verified in Section 9). A totally antisymmetric 3-tensor on the tangent space is exactly what is needed to add TOTALLY ANTISYMMETRIC TORSION to a metric connection without spoiling metric compatibility. Define, for a real parameter λ ,

$$\omega_{\text{Oct}}[\mu, a, b] = \omega_{\text{TriLC}}[\mu, a, b] + \lambda * m_{\text{Skew}}[a, b, c] * \text{frameTri}[[\mu, c]].$$

The added term is antisymmetric in (a, b) , so ω_{Oct} is still an $\mathfrak{so}(4, 4)$ -valued connection and is still metric compatible; but it is no longer torsion-free. Its torsion is totally antisymmetric and proportional to the octonion structure constants. This is the octonionic analogue of the Cartan-Schouten flat connections on a Lie group, and a totally antisymmetric torsion is precisely the kind that sources the Einstein-Cartan AXIAL torsion coupling of a spinor. What Section 20 computes is the torsion itself and its effect on the connection; the cubic $\gamma^a \gamma^b \gamma^c$ axial term is a further contraction that this notebook does not carry out, and the cell that builds the spinor coupling says so. At $\lambda = 0$ the connection reduces to the Levi-Civita connection in the triality frame.

```
ClearAll[frameTri,  $\eta$ TriFlat];
 $\eta$ TriFlat =  $\eta$ Tri; (* == sigma, proved in Section 9 *)
cfAssert["the triality bridge is ORTHOGONAL: Transpose[P] == Inverse[P]",
  cfZeroArrayQ[Transpose[triVecToSpin] - triSpinToVec]];
frameTri = cfSimpArray[frameCanonical . triVecToSpin];
cfAssert["BRIDGE 3 reproduces the SAME curved metric g",
  cfZeroArrayQ[frameTri .  $\eta$ TriFlat . Transpose[frameTri] - gCanonical]];
cfAssert["the triality tangent metric is sigma",  $\eta$ TriFlat ===  $\sigma$ ];
MatrixForm[frameTri]
```

```

(* --- the totally antisymmetric structure constants,
transported to the spinor basis ----- *)
(* The antisymmetrization is written out
over the six permutations of the three POSITIONS,    *)
(* with the standard signs. Writing it as
a sum over Permutations of the index VALUES would  *)
(* be wrong: Signature there is taken relative to sorted order,
which produces a totally      *)
(* SYMMETRIC object
instead. *)
ClearAll[mLowerTotallyAntisym, mSkewSpin];
mLowerTotallyAntisym = Table[
  (1/6) (mLower[[A1, B1, C1]] - mLower[[B1, A1, C1]] - mLower[[A1, C1, B1]]
    - mLower[[C1, B1, A1]] + mLower[[B1, C1, A1]] + mLower[[C1, A1, B1]]),
  {A1, 8}, {B1, 8}, {C1, 8}];
cfAssert["the antisymmetrized structure constants are totally antisymmetric",
cfZeroArrayQ[{
  Table[mLowerTotallyAntisym[[A1, B1, C1]] + mLowerTotallyAntisym[[B1, A1, C1]],
  {A1, 8}, {B1, 8}, {C1, 8}],
  Table[mLowerTotallyAntisym[[A1, B1, C1]] + mLowerTotallyAntisym[[A1, C1, B1]],
  {A1, 8}, {B1, 8}, {C1, 8}]]];
cfAssert["they agree with mLower on the seven imaginary directions",
cfZeroArrayQ[Table[mLowerTotallyAntisym[[A1, B1, C1]] - mLower[[A1, B1, C1]],
  {A1, 2, 8}, {B1, 2, 8}, {C1, 2, 8}]]];
(* transport all three indices through the triality bridge;
total antisymmetry is preserved *)
mSkewSpin = cfTimed["transport the skew structure constants to the spinor basis",
  Table[Sum[triSpinToVec[[a, A1]] triSpinToVec[[b, B1]] triSpinToVec[[c, C1]] ×
    mLowerTotallyAntisym[[A1, B1, C1]], {A1, 8}, {B1, 8}, {C1, 8}],
  {a, 8}, {b, 8}, {c, 8}];
cfAssert["mSkewSpin is totally antisymmetric",

  cfZeroArrayQ[{Table[mSkewSpin[[a, b, c]] + mSkewSpin[[b, a, c]], {a, 8}, {b, 8}, {c, 8}],
    Table[mSkewSpin[[a, b, c]] + mSkewSpin[[a, c, b]], {a, 8}, {b, 8}, {c, 8}]]];
{Dimensions[mSkewSpin], Count[Flatten[mSkewSpin], Except[0]], " non-zero components"}

```

```

(* --- first the Levi-
Civita connection in the triality frame (the lambda = 0 case) ----- *)
ClearAll[omegaTriLCMixed, omegaTriLC];
omegaTriLCMixed = cfTimed["BRIDGE 3 Levi-Civita part",
  cfSpinConnection[frameTri, X,  $\eta$ TriFlat, GammaCanonical]]];
omegaTriLC = cfLowerFirstFlat[omegaTriLCMixed,  $\eta$ TriFlat];
cfAssert["BRIDGE 3 (lambda=0): vielbein postulate holds identically",
  cfZeroArrayQ[cfVielbeinResidual[frameTri, X, GammaCanonical, omegaTriLCMixed]]];
cfAssert["BRIDGE 3 (lambda=0): omega[mu,a,b] == -omega[mu,b,a]",
  cfZeroArrayQ[Table[omegaTriLC[[mu, a, b]] + omegaTriLC[[mu, b, a]],
    {mu, 8}, {a, 8}, {b, 8}]]];
cfAssert[
  "BRIDGE 3 (lambda=0) is the constant triality conjugate of the canonical connection",
  cfZeroArrayQ[Table[omegaTriLCMixed[[mu]]
    - triSpinToVec.omegaCanonicalMixed[[mu]].triVecToSpin, {mu, 8}]]];
{Count[Flatten[omegaTriLC], Except[0]], " non-zero components"}

(* --- now the NEW octonionic connection
with totally antisymmetric torsion ----- *)
ClearAll[ $\lambda$ Oct, contortionOct, omegaOct, omegaOctMixed];
(* lambda is a free real torsion-strength parameter introduced here,
not in the original *)
contortionOct = Table[ $\lambda$ Oct Sum[mSkewSpin[[a, b, c]] frameTri[[mu, c]], {c, 8}],
  {mu, 8}, {a, 8}, {b, 8}];
omegaOct = cfSimpArray[omegaTriLC + contortionOct];
(* raise the first flat index again;
the triality tangent metric sigma is its own inverse *)
omegaOctMixed = Table[Sum[ $\eta$ TriFlat[[a, c]] omegaOct[[mu, c, b]], {c, 8}],
  {mu, 8}, {a, 8}, {b, 8}];
cfAssert[
  "BRIDGE 3: omegaOct[mu,a,b] == -omegaOct[mu,b,a] (still metric compatible)",
  cfZeroArrayQ[Table[omegaOct[[mu, a, b]] + omegaOct[[mu, b, a]], {mu, 8}, {a, 8}, {b, 8}]]];
cfAssert["BRIDGE 3: at lambda = 0 it reduces to the Levi-Civita connection",
  cfZeroArrayQ[(omegaOct /.  $\lambda$ Oct  $\rightarrow$  0) - omegaTriLC]];
{Count[Flatten[contortionOct], Except[0]], " non-zero contortion components"}

```

```

(* --- the torsion of the octonionic connection is non-
zero and totally antisymmetric ----- *)
ClearAll[torsionOct, torsionOctFlat];
torsionOct =
  cfTimed["BRIDGE 3 torsion 2-form", cfTorsion[frameTri, X, omegaOctMixed]];
cfAssert["BRIDGE 3: torsion vanishes at lambda = 0",
  cfZeroArrayQ[torsionOct /.  $\lambda_{Oct} \rightarrow 0$ ]];
cfAssert["BRIDGE 3: torsion is NOT zero for lambda != 0",
  ! TrueQ[cfZeroArrayQ[torsionOct /.  $\lambda_{Oct} \rightarrow 1$ ]]];
(* pull the torsion back to all-flat indices:  $T_{\{abc\}} =$ 
 $\eta_{\{ad\}} T^d_{\{\mu \nu\}} e_b^{\mu} e_c^{\nu}$  *)
torsionOctFlat = cfTimed["BRIDGE 3 torsion with all indices flat",
  Module[{cofr = cfSimpArray[Inverse[frameTri]]},
    cfSimpArray @ Table[
      Sum[ $\eta_{TriFlat}[[a, d]]$  torsionOct[[d, mu, nu]] cofr[[b, mu]] cofr[[c, nu]],
      {d, 8}, {mu, 8}, {nu, 8}], {a, 8}, {b, 8}, {c, 8}]]];
cfAssert["BRIDGE 3: the flat torsion  $T[a,b,c]$  is TOTALLY ANTISYMMETRIC",
  cfZeroArrayQ[{Table[torsionOctFlat[[a, b, c]] + torsionOctFlat[[b, a, c]],
    {a, 8}, {b, 8}, {c, 8}],
    Table[torsionOctFlat[[a, b, c]] + torsionOctFlat[[a, c, b]],
    {a, 8}, {b, 8}, {c, 8}]]];
cfAssert["BRIDGE 3:  $T[a,b,c] == -2 \lambda$  mSkewSpin[a,b,c]",
  cfZeroArrayQ[Table[torsionOctFlat[[a, b, c]] + 2  $\lambda_{Oct}$  mSkewSpin[[a, b, c]],
    {a, 8}, {b, 8}, {c, 8}]]];
{Count[Flatten[torsionOctFlat], Except[0]], " non-zero flat torsion components"}

(* --- COMPARISON 3:
the difference from the canonical connection is exactly the contortion - *)
ClearAll[deltaOct, contortionOctMixed];
contortionOctMixed = Table[Sum[ $\eta_{TriFlat}[[a, c]]$  contortionOct[[mu, c, b]], {c, 8}],
  {mu, 8}, {a, 8}, {b, 8}];
deltaOct = cfSimpArray[Table[
  omegaOctMixed[[mu]] - triSpinToVec . omegaCanonicalMixed[[mu]] . triVecToSpin, {mu, 8}]];
cfAssert[
  "COMPARISON 3: omegaOct - (triality conjugate of omegaCanonical) == contortion",
  cfZeroArrayQ[deltaOct - contortionOctMixed]];
cfAssert[
  "COMPARISON 3: the difference is linear in lambda and vanishes at lambda = 0",
  cfZeroArrayQ[deltaOct /.  $\lambda_{Oct} \rightarrow 0$ ]];
{Count[Flatten[deltaOct], Except[0]], " non-zero components in the difference"}

```

The effect on the spinor covariant derivative. One thing must be got right here, and it is easy to get wrong. cfSpinMatrix of Section 17 contracts the flat indices of a connection against the Dirac matrices T16, and that is correct only when those flat indices are the eta4488 VECTOR indices, because T16 obeys $\text{Anticommutator}[T16[A], T16[B]] == 2 \text{eta4488}[[A+1, B+1]] \text{ID16}$. Bridge 3's flat index is not a vector index: it is a triality-spinor index whose tangent metric is sigma. Feeding omegaOct into cfSpinMatrix would therefore build the spin representative of a

different object.

The Dirac matrices that match this frame are the triality transports

$$T16Tri[a] = \text{Sum over } A \text{ of } triSpinToVec[[a,A]] T16[A-1]$$

and they obey $\text{Anticommutator}[T16Tri[a], T16Tri[b]] == 2 \text{ sigma}[[a,b]] \text{ ID16}$, which is exactly the statement that the triality bridge carries η_{4488} onto sigma . We verify that, and then build Bridge 3's spinor connection with them.

```
ClearAll[T16Tri, cfSpinMatrixTri];
T16Tri[a_Integer] := T16Tri[a] = Sum[triSpinToVec[[a, A1]] T16[A1 - 1], {A1, 8}];
cfAssert["THE TRIALITY DIRAC MATRICES OBEY THE sigma
  CLIFFORD RELATION: {T16Tri[a],T16Tri[b]} == 2 sigma[[a,b]] ID16",
  cfZeroArrayQ[Table[T16Tri[a] . T16Tri[b] + T16Tri[b] . T16Tri[a]
    - 2 \eta TriFlat[[a, b]] ID16, {a, 8}, {b, 8}]]];
cfAssert["they are NOT the vector-frame Dirac matrices, so the distinction is real",
  ! TrueQ[cfZeroArrayQ[Table[T16Tri[a] - T16[a - 1], {a, 8}]]]];
cfSpinMatrixTri[om_, mu_Integer] :=
  (1/8) Sum[om[[mu, a, b]] (T16Tri[a] . T16Tri[b] - T16Tri[b] . T16Tri[a]), {a, 8}, {b, 8}];
```

```

(* --- the effect on the spinor covariant
   derivative: the torsion's so(4,4) piece ----- *)
(* Dcov picks up (lambda/8) mSkew[a,b,c]
   frameTri[[mu,c]] Commutator[gammaTri^a, gammaTri^b]. *)
(* Count the gammas: that expression is QUADRATIC in gamma,
   not cubic. The third structure- *)
(* constant index c is contracted against the FRAME,
   frameTri[[mu,c]], not against a third *)
(* gamma. What this cell exhibits is therefore
   the torsion's contribution to the so(4,4) *)
(* part of the spinor connection,
   generated by the split-octonion multiplication itself *)
(* rather than put in by
   hand. *)
(* The familiar Einstein-Cartan AXIAL term,
   T[a,b,c] gamma^a gamma^b gamma^c, is the genuinely *)
(* cubic object,
   and it appears only after this connection piece is contracted with the *)
(* gamma^mu of the Dirac operator. This
   notebook does not perform that contraction, so it *)
(* does not claim to have displayed the axial
   coupling. *)
ClearAll[GammaSpinOct, GammaSpinOctExtra];
GammaSpinOct = cFTimed["BRIDGE 3 16x16 spin-connection matrices",
  Table[cfSimpArray[cfSpinMatrixTri[omegaOct, mu]], {mu, 8}]];
GammaSpinOctExtra = cFTimed["BRIDGE 3 extra spinor coupling from the torsion",
  Table[cfSimpArray[cfSpinMatrixTri[contortionOct, mu]], {mu, 8}]];
(* consistency: at lambda =
  0 the Bridge 3 spinor connection must be the SAME 16x16 matrix as *)
(* the canonical one,
   because the two frames differ only by a constant O(4,4) rotation. *)
cfAssert["at lambda = 0 the Bridge 3 spinor connection equals the canonical one",
  cfZeroArrayQ[Table[(GammaSpinOct[[mu]] /. lambdaOct -> 0) - GammaSpinCanonical[[mu]], {mu, 8}]]];
cfAssert["the extra spinor coupling vanishes at lambda = 0",
  cfZeroArrayQ[GammaSpinOctExtra /. lambdaOct -> 0]];
cfAssert["the extra spinor coupling is non-zero for lambda != 0",
  ! TrueQ[cfZeroArrayQ[GammaSpinOctExtra /. lambdaOct -> 1]]];
cfAssert[
  "the extra spinor coupling is still block diagonal (type-1 + type-2 preserved)",
  cfZeroArrayQ[Table[GammaSpinOctExtra[[mu]]
    - ArrayFlatten[{{GammaSpinOctExtra[[mu]][[1 ;; 8, 1 ;; 8]], 0},
      {0, GammaSpinOctExtra[[mu]][[9 ;; 16, 9 ;; 16]]}], {mu, 8}]]];
{Count[Flatten[GammaSpinOctExtra], Except[0]], " non-zero entries in the extra coupling"}

```

21. Master comparison of the four spin connections

Everything computed above, collected in one place. For each connection we record the frame field it comes from, the flat tangent metric it uses, whether the curved metric g is unchanged (it is, for all four: they are all bridges for the SAME geometry), whether the connection is metric compatible, whether its torsion vanishes, and how it differs from the canonical one.

```
ClearAll[cfConnectionSummary];
cfConnectionSummary = {
  {"connection", "frame field",
    "flat metric", "same g?", "antisymmetric?", "torsion zero?",
    "difference from canonical"},
  {"omegaCanonical", "frameCanonical (diagonal)",
    "eta4488", "yes (reference)", "yes", "yes",
    "-- (reference)"},
  {"omegaBoost", "frameCanonical . Lambda(x)", "eta4488", "yes", "yes", "yes",
    "inhomogeneous gauge term -D[theta,x_mu] K"},
  {"omegaNull", "frameCanonical . U", "etaNull == sigma", "yes", "yes", "yes",
    "constant similarity only: U omega U"},
  {"omegaOct", "frameCanonical . triVecToSpin", "etaTri == sigma", "yes", "yes",
    "NO: torsion == -2 lambda mSkew",
    "contortion lambda mSkew[a,b,c] frameTri[[mu,c]]"}];
Grid[cfConnectionSummary, Frame -> All, Alignment -> Left,
  Background -> {None, {LightGray, None, None, None, None}}]

(* --- numerical
  fingerprint: the count of non-zero components of each object ----- *)
Grid[{
  {"object", "non-zero components"},
  {"Gamma (Christoffel)", Count[Flatten[GammaCanonical], Except[0]]},
  {"omegaCanonical", Count[Flatten[omegaCanonical], Except[0]]},
  {"omegaBoost", Count[Flatten[omegaBoost], Except[0]]},
  {"omegaNull", Count[Flatten[omegaNull], Except[0]]},
  {"omegaTriLC (lambda=0)", Count[Flatten[omegaTriLC], Except[0]]},
  {"contortionOct", Count[Flatten[contortionOct], Except[0]]},
  {"RiemannCanonical", Count[Flatten[RiemannCanonical], Except[0]]},
  {"RiemannBoost", Count[Flatten[RiemannBoost], Except[0]]},
  {"RiemannNull", Count[Flatten[RiemannNull], Except[0]]},
  {"torsionOctFlat", Count[Flatten[torsionOctFlat], Except[0]]}
}, Frame -> All, Alignment -> Left]
```

```

(* --- the three comparisons,
side by side ----- *)
Grid[{
  {"comparison", "statement proved above"},
  {"BRIDGE 1 vs canonical",
   "omegaBoost_mu == M omegaCanonical_mu
    Inverse[M] - D[M,x_mu] Inverse[M], M = Transpose[Lambda];"
   "<> \" the whole difference is -D[theta,x_mu]
    K, one scalar gradient times one fixed generator"},
  {"BRIDGE 2 vs canonical",
   "omegaNull_mu == U omegaCanonical_mu U   exactly, with no inhomogeneous term"},
  {"BRIDGE 3 vs canonical",
   "omegaOct_mu == Inverse[P] omegaCanonical_mu
    P + lambda mSkew . frameTri, P = triVecToSpin"},
  {"curvature",
   "R transforms covariantly under
    bridges 1 and 2, so all three describe the same geometry; "
   "<> \"bridge 3 adds a torsion-dependent piece"},
  {"Ricci scalar", "identical for the canonical frame and bridges 1 and 2"},
  {"torsion",
   "zero for the canonical connection
    and for bridges 1 and 2; -2 lambda mSkew for bridge 3"}
}, Frame → All, Alignment → Left]

(* --- final tally of every assertion made
in this notebook ----- *)
Print["identities accepted on numerical evidence alone : ", $cfNumericCertificates,
      " (stage 3 returned True)"];
Print["non-vanishing witnesses : ", $cfNonVanishingWitnesses,
      " (stage 3 returned False inside a ! TrueQ[...] assertion);"];
Print[
  "
a sample point with a non-zero",
  " value is a complete proof of non-vanishing)"];
cfAssertSummary[]

```